

CHAPTER

2

Regular Expressions, Text Normalization, Edit Distance

```

User: I am unhappy.
ELIZA: DO YOU THINK COMING HERE WILL HELP YOU NOT TO BE UNHAPPY
User: I need some help, that much seems certain.
ELIZA: WHAT WOULD IT MEAN TO YOU IF YOU GOT SOME HELP
User: Perhaps I could learn to get along with my mother.
ELIZA: TELL ME MORE ABOUT YOUR FAMILY
User: My mother takes care of me.
ELIZA: WHO ELSE IN YOUR FAMILY TAKES CARE OF YOU
User: My father.
ELIZA: YOUR FATHER
User: You are like my father in some ways.

```

Weizenbaum (1966)

ELIZA

The dialogue above is from **ELIZA**, an early natural language processing system that could carry on a limited conversation with a user by imitating the responses of a Rogerian psychotherapist (Weizenbaum, 1966). ELIZA is a surprisingly simple program that uses pattern matching to recognize phrases like “I need X” and translate them into suitable outputs like “What would it mean to you if you got X?”. This simple technique succeeds in this domain because ELIZA doesn’t actually need to *know* anything to mimic a Rogerian psychotherapist. As Weizenbaum notes, this is one of the few dialogue genres where listeners can act as if they know nothing of the world. ELIZA’s mimicry of human conversation was remarkably successful: many people who interacted with ELIZA came to believe that it really *understood* them and their problems, many continued to believe in ELIZA’s abilities even after the program’s operation was explained to them (Weizenbaum, 1976), and even today such **chatbots** are a fun diversion.

chatbots

Of course modern conversational agents are much more than a diversion; they can answer questions, book flights, or find restaurants, functions for which they rely on a much more sophisticated understanding of the user’s intent, as we will see in Chapter 15. Nonetheless, the simple pattern-based methods that powered ELIZA and other chatbots play a crucial role in natural language processing.

We’ll begin with the most important tool for describing text patterns: the **regular expression**. Regular expressions can be used to specify strings we might want to extract from a document, from transforming “I need X” in ELIZA above, to defining strings like \$199 or \$24.99 for extracting tables of prices from a document.

text
normalization

We’ll then turn to a set of tasks collectively called **text normalization**, in which regular expressions play an important part. Normalizing text means converting it to a more convenient, standard form. For example, most of what we are going to do with language relies on first separating out or **tokenizing** words from running text, the task of **tokenization**. English words are often separated from each other by whitespace, but whitespace is not always sufficient. *New York* and *rock ‘n’ roll* are sometimes treated as large words despite the fact that they contain spaces, while sometimes we’ll need to separate *I’m* into the two words *I* and *am*. For processing tweets or texts we’ll need to tokenize **emoticons** like :) or **hashtags** like #n1proc.

tokenization

Some languages, like Japanese, don't have spaces between words, so word tokenization becomes more difficult.

lemmatization

Another part of text normalization is **lemmatization**, the task of determining that two words have the same root, despite their surface differences. For example, the words *sang*, *sung*, and *sings* are forms of the verb *sing*. The word *sing* is the common *lemma* of these words, and a **lemmatizer** maps from all of these to *sing*.

stemming

Lemmatization is essential for processing morphologically complex languages like Arabic. **Stemming** refers to a simpler version of lemmatization in which we mainly just strip suffixes from the end of the word. Text normalization also includes **sen-**

sentence
segmentation

tence segmentation: breaking up a text into individual sentences, using cues like periods or exclamation points.

Finally, we'll need to compare words and other strings. We'll introduce a metric called **edit distance** that measures how similar two strings are based on the number of edits (insertions, deletions, substitutions) it takes to change one string into the other. Edit distance is an algorithm with applications throughout language processing, from spelling correction to speech recognition to coreference resolution.

2.1 Regular Expressions

regular
expression

One of the unsung successes in standardization in computer science has been the **regular expression** (often shortened to **regex**), a language for specifying text search strings. This practical language is used in every computer language, word processor, and text processing tools like the Unix tools `grep` or Emacs. Formally, a regular expression is an algebraic notation for characterizing a set of strings. Regular expressions are particularly useful for searching in texts, when we have a **pattern** to search for and a **corpus** of texts to search through. A regular expression search function will search through the corpus, returning all texts that match the pattern. The corpus can be a single document or a collection. For example, the Unix command-line tool `grep` takes a regular expression and returns every line of the input document that matches the expression.

corpus

A search can be designed to return every match on a line, if there are more than one, or just the first match. In the following examples we generally underline the exact part of the pattern that matches the regular expression and show only the first match. We'll show regular expressions delimited by slashes but note that slashes are *not* part of the regular expressions.

Regular expressions come in many variants. We'll be describing **extended regular expressions**; different regular expression parsers may only recognize subsets of these, or treat some expressions slightly differently. Using an online regular expression tester is a handy way to test out your expressions and explore these variations.

2.1.1 Basic Regular Expression Patterns

concatenation

The simplest kind of regular expression is a sequence of simple characters; putting characters in sequence is called **concatenation**. To search for *woodchuck*, we type `/woodchuck/`. The expression `/Buttercup/` matches any string containing the substring *Buttercup*; `grep` with that expression would return the line *I'm called little Buttercup*. The search string can consist of a single character (like `/!/`) or a sequence of characters (like `/url/`) (see Fig. 2.1).

Regular expressions are **case sensitive**; lower case `/s/` is distinct from upper

Regex	Example Patterns Matched
/woodchucks/	“interesting links to woodchucks and lemurs”
/a/	“Mary Ann stopped by Mona’s”
/!/	“You’ve left the burglar behind again!” said Nori

Figure 2.1 Some simple regex searches.

case /S/ (/s/ matches a lower case *s* but not an upper case *S*). This means that the pattern /woodchucks/ will not match the string *Woodchucks*. We can solve this problem with the use of the square braces [and]. The string of characters inside the braces specifies a **disjunction** of characters to match. For example, Fig. 2.2 shows that the pattern /[wW]/ matches patterns containing either *w* or *W*.

Regex	Match	Example Patterns
/[wW]oodchuck/	Woodchuck or woodchuck	“ <u>W</u> oodchuck”
/[abc]/	‘a’, ‘b’, or ‘c’	“In uomini, in soldati”
/[1234567890]/	any digit	“plenty of <u>7</u> to 5”

Figure 2.2 The use of the brackets [] to specify a disjunction of characters.

The regular expression /[1234567890]/ specifies any single digit. While such classes of characters as digits or letters are important building blocks in expressions, they can get awkward (e.g., it’s inconvenient to specify

/[ABCDEFGHIJKLMNOPQRSTUVWXYZ]/

to mean “any capital letter”). In cases where there is a well-defined sequence associated with a set of characters, the brackets can be used with the dash (–) to specify any one character in a **range**. The pattern /[2–5]/ specifies any one of the characters 2, 3, 4, or 5. The pattern /[b–g]/ specifies one of the characters *b*, *c*, *d*, *e*, *f*, or *g*. Some other examples are shown in Fig. 2.3.

Regex	Match	Example Patterns Matched
/[A–Z]/	an upper case letter	“we should call it ‘ <u>D</u> renched Blossoms’ ”
/[a–z]/	a lower case letter	“ <u>m</u> y beans were impatient to be hoed!”
/[0–9]/	a single digit	“Chapter <u>1</u> : Down the Rabbit Hole”

Figure 2.3 The use of the brackets [] plus the dash – to specify a range.

The square braces can also be used to specify what a single character *cannot* be, by use of the caret ^. If the caret ^ is the first symbol after the open square brace [, the resulting pattern is negated. For example, the pattern /[[^]a]/ matches any single character (including special characters) except *a*. This is only true when the caret is the first symbol after the open square brace. If it occurs anywhere else, it usually stands for a caret; Fig. 2.4 shows some examples.

Regex	Match (single characters)	Example Patterns Matched
/[[^] A–Z]/	not an upper case letter	“Oyfn pripetchik”
/[[^] Ss]/	neither ‘S’ nor ‘s’	“ <u>I</u> have no exquisite reason for’t”
/[[^] .]/	not a period	“ <u>o</u> ur resident Djinn”
/[e [^]]/	either ‘e’ or ‘ [^] ’	“look up <u>^</u> now”
/a [^] b/	the pattern ‘a [^] b’	“look up a [^] b now”

Figure 2.4 The caret ^ for negation or just to mean ^. See below re: the backslash for escaping the period.

How can we talk about optional elements, like an optional *s* in *woodchuck* and *woodchucks*? We can’t use the square brackets, because while they allow us to say

“s or S”, they don’t allow us to say “s or nothing”. For this we use the question mark `/?/`, which means “the preceding character or nothing”, as shown in Fig. 2.5.

Regex	Match	Example Patterns Matched
<code>/woodchucks?/</code>	woodchuck or woodchucks	<u>“woodchuck”</u>
<code>/colou?r/</code>	color or colour	<u>“color”</u>

Figure 2.5 The question mark `?` marks optionality of the previous expression.

We can think of the question mark as meaning “zero or one instances of the previous character”. That is, it’s a way of specifying how many of something that we want, something that is very important in regular expressions. For example, consider the language of certain sheep, which consists of strings that look like the following:

```
baa!
baaa!
baaaa!
baaaaa!
...
```

Kleene *

This language consists of strings with a *b*, followed by at least two *a*’s, followed by an exclamation point. The set of operators that allows us to say things like “some number of *as*” are based on the asterisk or `*`, commonly called the **Kleene *** (generally pronounced “cleany star”). The Kleene star means “zero or more occurrences of the immediately previous character or regular expression”. So `/a*/` means “any string of zero or more *as*”. This will match *a* or *aaaaaa*, but it will also match the empty string at the start of *Off Minor* since the string *Off Minor* starts with zero *a*’s. So the regular expression for matching one or more *a* is `/aa*/`, meaning one *a* followed by zero or more *as*. More complex patterns can also be repeated. So `/[ab]*/` means “zero or more *a*’s or *b*’s” (not “zero or more right square braces”). This will match strings like *aaaa* or *ababab* or *bbbb*.

For specifying multiple digits (useful for finding prices) we can extend `/[0-9]/`, the regular expression for a single digit. An integer (a string of digits) is thus `/[0-9][0-9]*/`. (Why isn’t it just `/[0-9]*/`?)

Kleene +

Sometimes it’s annoying to have to write the regular expression for digits twice, so there is a shorter way to specify “at least one” of some character. This is the **Kleene +**, which means “one or more occurrences of the immediately preceding character or regular expression”. Thus, the expression `/[0-9]+/` is the normal way to specify “a sequence of digits”. There are thus two ways to specify the sheep language: `/baaa*/` or `/baa+/`.

One very important special character is the period (`/./`), a **wildcard** expression that matches any single character (*except* a carriage return), as shown in Fig. 2.6.

Regex	Match	Example Matches
<code>/beg.n/</code>	any character between <i>beg</i> and <i>n</i>	<u>begin</u> , <u>beg’n</u> , <u>begun</u>

Figure 2.6 The use of the period `.` to specify any character.

The wildcard is often used together with the Kleene star to mean “any string of characters”. For example, suppose we want to find any line in which a particular word, for example, *aardvark*, appears twice. We can specify this with the regular expression `/aardvark.*aardvark/`.

anchors

anchors are special characters that anchor regular expressions to particular places

in a string. The most common anchors are the caret `^` and the dollar sign `$`. The caret `^` matches the start of a line. The pattern `/^The/` matches the word *The* only at the start of a line. Thus, the caret `^` has three uses: to match the start of a line, to indicate a negation inside of square brackets, and just to mean a caret. (What are the contexts that allow `grep` or Python to know which function a given caret is supposed to have?) The dollar sign `$` matches the end of a line. So the pattern `␣$` is a useful pattern for matching a space at the end of a line, and `/^The dog\.$/` matches a line that contains only the phrase *The dog*. (We have to use the backslash here since we want the `.` to mean “period” and not the wildcard.)

Regex	Match
<code>^</code>	start of line
<code>\$</code>	end of line
<code>\b</code>	word boundary
<code>\B</code>	non-word boundary

Figure 2.7 Anchors in regular expressions.

There are also two other anchors: `\b` matches a word boundary, and `\B` matches a non-boundary. Thus, `/\bthe\b/` matches the word *the* but not the word *other*. More technically, a “word” for the purposes of a regular expression is defined as any sequence of digits, underscores, or letters; this is based on the definition of “words” in programming languages. For example, `/\b99\b/` will match the string *99* in *There are 99 bottles of beer on the wall* (because *99* follows a space) but not *99* in *There are 299 bottles of beer on the wall* (since *99* follows a number). But it will match *99* in *\$99* (since *99* follows a dollar sign (`$`), which is not a digit, underscore, or letter).

2.1.2 Disjunction, Grouping, and Precedence

Suppose we need to search for texts about pets; perhaps we are particularly interested in cats and dogs. In such a case, we might want to search for either the string *cat* or the string *dog*. Since we can’t use the square brackets to search for “cat or dog” (why can’t we say `/[catdog]/?`), we need a new operator, the **disjunction** operator, also called the **pipe** symbol `|`. The pattern `/cat|dog/` matches either the string *cat* or the string *dog*.

Sometimes we need to use this disjunction operator in the midst of a larger sequence. For example, suppose I want to search for information about pet fish for my cousin David. How can I specify both *guppy* and *guppies*? We cannot simply say `/guppy|ies/`, because that would match only the strings *guppy* and *ies*. This is because sequences like *guppy* take **precedence** over the disjunction operator `|`. To make the disjunction operator apply only to a specific pattern, we need to use the parenthesis operators `(` and `)`. Enclosing a pattern in parentheses makes it act like a single character for the purposes of neighboring operators like the pipe `|` and the Kleene*. So the pattern `/gupp(y|ies)/` would specify that we meant the disjunction only to apply to the suffixes *y* and *ies*.

The parenthesis operator `(` is also useful when we are using counters like the Kleene*. Unlike the `|` operator, the Kleene* operator applies by default only to a single character, not to a whole sequence. Suppose we want to match repeated instances of a string. Perhaps we have a line that has column labels of the form *Column 1 Column 2 Column 3*. The expression `/Column␣[0-9]+␣*/` will not match any number of columns; instead, it will match a single column followed by

any number of spaces! The star here applies only to the space `␣` that precedes it, not to the whole sequence. With the parentheses, we could write the expression `/(Column␣[0-9]+␣*)*/` to match the word *Column*, followed by a number and optional spaces, the whole pattern repeated zero or more times.

operator
precedence

This idea that one operator may take precedence over another, requiring us to sometimes use parentheses to specify what we mean, is formalized by the **operator precedence hierarchy** for regular expressions. The following table gives the order of RE operator precedence, from highest precedence to lowest precedence.

Parenthesis	()
Counters	* + ? {}
Sequences and anchors	the ^my end\$
Disjunction	

Thus, because counters have a higher precedence than sequences, `/the*/` matches *theeee* but not *thethe*. Because sequences have a higher precedence than disjunction, `/the|any/` matches *the* or *any* but not *thany* or *theny*.

Patterns can be ambiguous in another way. Consider the expression `/[a-z]*/` when matching against the text *once upon a time*. Since `/[a-z]*/` matches zero or more letters, this expression could match nothing, or just the first letter *o*, *on*, *onc*, or *once*. In these cases regular expressions always match the *largest* string they can; we say that patterns are **greedy**, expanding to cover as much of a string as they can.

greedy
non-greedy

There are, however, ways to enforce **non-greedy** matching, using another meaning of the `?` qualifier. The operator `*?` is a Kleene star that matches as little text as possible. The operator `+?` is a Kleene plus that matches as little text as possible.

2.1.3 A Simple Example

Suppose we wanted to write a RE to find cases of the English article *the*. A simple (but incorrect) pattern might be:

```
/the/
```

One problem is that this pattern will miss the word when it begins a sentence and hence is capitalized (i.e., *The*). This might lead us to the following pattern:

```
/[tT]he/
```

But we will still incorrectly return texts with **the** embedded in other words (e.g., *other* or *theology*). So we need to specify that we want instances with a word boundary on both sides:

```
/\b[tT]he\b/
```

Suppose we wanted to do this without the use of `/\b/`. We might want this since `/\b/` won't treat underscores and numbers as word boundaries; but we might want to find *the* in some context where it might also have underlines or numbers nearby (*the_* or *the25*). We need to specify that we want instances in which there are no alphabetic letters on either side of the *the*:

```
/[^a-zA-Z][tT]he[^a-zA-Z]/
```

But there is still one more problem with this pattern: it won't find the word *the* when it begins a line. This is because the regular expression `[^a-zA-Z]`, which

we used to avoid embedded instances of *the*, implies that there must be some single (although non-alphabetic) character before the *the*. We can avoid this by specifying that before the *the* we require *either* the beginning-of-line or a non-alphabetic character, and the same at the end of the line:

```
/(^|[^a-zA-Z])[tT]he([a-zA-Z]|$)/
```

false positives
false negatives

The process we just went through was based on fixing two kinds of errors: **false positives**, strings that we incorrectly matched like *other* or *there*, and **false negatives**, strings that we incorrectly missed, like *The*. Addressing these two kinds of errors comes up again and again in implementing speech and language processing systems. Reducing the overall error rate for an application thus involves two antagonistic efforts:

- Increasing **precision** (minimizing false positives)
- Increasing **recall** (minimizing false negatives)

We'll come back to precision and recall with more precise definitions in Chapter 4.

2.1.4 More Operators

Figure 2.8 shows some aliases for common ranges, which can be used mainly to save typing. Besides the Kleene $*$ and Kleene $+$ we can also use explicit numbers as counters, by enclosing them in curly brackets. The regular expression `/ {3} /` means “exactly 3 occurrences of the previous character or expression”. So `/a\.{24}z/` will match *a* followed by 24 dots followed by *z* (but not *a* followed by 23 or 25 dots followed by a *z*).

Regex	Expansion	Match	First Matches
<code>\d</code>	<code>[0-9]</code>	any digit	Party_of_5
<code>\D</code>	<code>[^0-9]</code>	any non-digit	Blue_moon
<code>\w</code>	<code>[a-zA-Z0-9_]</code>	any alphanumeric/underscore	Daiyu
<code>\W</code>	<code>[^\w]</code>	a non-alphanumeric	!!!!
<code>\s</code>	<code>[\r\t\n\f]</code>	whitespace (space, tab)	in_Concord
<code>\S</code>	<code>[^\s]</code>	Non-whitespace	in_Concord

Figure 2.8 Aliases for common sets of characters.

A range of numbers can also be specified. So `/ {n,m} /` specifies from *n* to *m* occurrences of the previous char or expression, and `/ {n,} /` means at least *n* occurrences of the previous expression. REs for counting are summarized in Fig. 2.9.

Regex	Match
<code>*</code>	zero or more occurrences of the previous char or expression
<code>+</code>	one or more occurrences of the previous char or expression
<code>?</code>	zero or one occurrence of the previous char or expression
<code>{n}</code>	exactly <i>n</i> occurrences of the previous char or expression
<code>{n,m}</code>	from <i>n</i> to <i>m</i> occurrences of the previous char or expression
<code>{n,}</code>	at least <i>n</i> occurrences of the previous char or expression
<code>{,m}</code>	up to <i>m</i> occurrences of the previous char or expression

Figure 2.9 Regular expression operators for counting.

Finally, certain special characters are referred to by special notation based on the backslash (`\`) (see Fig. 2.10). The most common of these are the **newline** character

`\n` and the **tab** character `\t`. To refer to characters that are special themselves (like `.`, `*`, `[`, and `\`), precede them with a backslash, (i.e., `/\./`, `/\*/`, `/\[`, and `/\[`).

Regex	Match	First Patterns Matched
<code>*</code>	an asterisk “ <code>*</code> ”	“ <code>K_A*P*L*A*N</code> ”
<code>\.</code>	a period “ <code>.</code> ”	“ <code>Dr. Livingston, I presume</code> ”
<code>\?</code>	a question mark	“ <code>Why don’t they come and lend a hand?</code> ”
<code>\n</code>	a newline	
<code>\t</code>	a tab	

Figure 2.10 Some characters that need to be backslashed.

2.1.5 A More Complex Example

Let’s try out a more significant example of the power of REs. Suppose we want to build an application to help a user buy a computer on the Web. The user might want “any machine with at least 6 GHz and 500 GB of disk space for less than \$1000”. To do this kind of retrieval, we first need to be able to look for expressions like *6 GHz* or *500 GB* or *Mac* or *\$999.99*. In the rest of this section we’ll work out some simple regular expressions for this task.

First, let’s complete our regular expression for prices. Here’s a regular expression for a dollar sign followed by a string of digits:

```
/[$0-9]+/
```

Note that the `$` character has a different function here than the end-of-line function we discussed earlier. Most regular expression parsers are smart enough to realize that `$` here doesn’t mean end-of-line. (As a thought experiment, think about how regex parsers might figure out the function of `$` from the context.)

Now we just need to deal with fractions of dollars. We’ll add a decimal point and two digits afterwards:

```
/[$0-9]+\.[0-9][0-9]/
```

This pattern only allows *\$199.99* but not *\$199*. We need to make the cents optional and to make sure we’re at a word boundary:

```
/(^|\W)$[0-9]+(\.[0-9][0-9])?\b/
```

One last catch! This pattern allows prices like *\$199999.99* which would be far too expensive! We need to limit the dollars:

```
/(^|\W)$[0-9]{0,3}(\.[0-9][0-9])?\b/
```

How about disk space? We’ll need to allow for optional fractions again (*5.5 GB*); note the use of `?` for making the final `s` optional, and the use of `/_*` to mean “zero or more spaces” since there might always be extra spaces lying around:

```
/\b[0-9]+(\.[0-9]+)?_*(GB|[Gg]igabytes?)\b/
```

Modifying this regular expression so that it only matches more than 500 GB is left as an exercise for the reader.

2.1.6 Substitution, Capture Groups, and ELIZA

substitution An important use of regular expressions is in **substitutions**. For example, the substitution operator `s/regexp1/pattern/` used in Python and in Unix commands like `vim` or `sed` allows a string characterized by a regular expression to be replaced by another string:

```
s/colour/color/
```

It is often useful to be able to refer to a particular subpart of the string matching the first pattern. For example, suppose we wanted to put angle brackets around all integers in a text, for example, changing *the 35 boxes* to *the <35> boxes*. We'd like a way to refer to the integer we've found so that we can easily add the brackets. To do this, we put parentheses (and) around the first pattern and use the **number** operator `\1` in the second pattern to refer back. Here's how it looks:

```
s/([0-9]+)/<\1>/
```

The parenthesis and number operators can also specify that a certain string or expression must occur twice in the text. For example, suppose we are looking for the pattern “the Xer they were, the Xer they will be”, where we want to constrain the two X's to be the same string. We do this by surrounding the first X with the parenthesis operator, and replacing the second X with the number operator `\1`, as follows:

```
/the (.*?)er they were, the \1er they will be/
```

Here the `\1` will be replaced by whatever string matched the first item in parentheses. So this will match *the bigger they were, the bigger they will be* but not *the bigger they were, the faster they will be*.

capture group This use of parentheses to store a pattern in memory is called a **capture group**. Every time a capture group is used (i.e., parentheses surround a pattern), the resulting match is stored in a numbered **register**. If you match two different sets of parentheses, `\2` means whatever matched the *second* capture group. Thus

```
/the (.*?)er they (.*), the \1er we \2/
```

will match *the faster they ran, the faster we ran* but not *the faster they ran, the faster we ate*. Similarly, the third capture group is stored in `\3`, the fourth is `\4`, and so on.

non-capturing group Parentheses thus have a double function in regular expressions; they are used to group terms for specifying the order in which operators should apply, and they are used to capture something in a register. Occasionally we might want to use parentheses for grouping, but don't want to capture the resulting pattern in a register. In that case we use a **non-capturing group**, which is specified by putting the special commands `?:` after the open parenthesis, in the form `(?: pattern )`.

```
/(?:some|a few) (people|cats) like some \1/
```

will match *some cats like some cats* but not *some cats like some some*.

Substitutions and capture groups are very useful in implementing simple chatbots like ELIZA (Weizenbaum, 1966). Recall that ELIZA simulates a Rogerian psychologist by carrying on conversations like the following:

```

User1:   Men are all alike.
ELIZA1: IN WHAT WAY
User2:   They're always bugging us about something or other.
ELIZA2: CAN YOU THINK OF A SPECIFIC EXAMPLE
User3:   Well, my boyfriend made me come here.
ELIZA3: YOUR BOYFRIEND MADE YOU COME HERE
User4:   He says I'm depressed much of the time.
ELIZA4: I AM SORRY TO HEAR YOU ARE DEPRESSED

```

ELIZA works by having a series or cascade of regular expression substitutions each of which matches and changes some part of the input lines. Input lines are first uppercased. The first substitutions then change all instances of *MY* to *YOUR*, and *I'M* to *YOU ARE*, and so on. The next set of substitutions matches and replaces other patterns in the input. Here are some examples:

```

s/. * I'M (depressed|sad) . */I AM SORRY TO HEAR YOU ARE \1/
s/. * I AM (depressed|sad) . */WHY DO YOU THINK YOU ARE \1/
s/. * all . */IN WHAT WAY/
s/. * always . */CAN YOU THINK OF A SPECIFIC EXAMPLE/

```

Since multiple substitutions can apply to a given input, substitutions are assigned a rank and applied in order. Creating patterns is the topic of Exercise 2.3, and we return to the details of the ELIZA architecture in Chapter 15.

2.1.7 Lookahead Assertions

Finally, there will be times when we need to predict the future: look ahead in the text to see if some pattern matches, but not advance the match cursor, so that we can then deal with the pattern if it occurs.

lookahead These **lookahead** assertions make use of the (? syntax that we saw in the previous section for non-capture groups. The operator (?= *pattern*) is true if *pattern* occurs, but is **zero-width**, i.e. the match pointer doesn't advance. The operator (?! *pattern*) only returns true if a pattern does not match, but again is zero-width and doesn't advance the cursor. Negative lookahead is commonly used when we are parsing some complex pattern but want to rule out a special case. For example suppose we want to match, at the beginning of a line, any single word that doesn't start with "Volcano". We can use negative lookahead to do this:

```
/^(?!Volcano)[A-Za-z]+/
```

2.2 Words

corpus
corpora

Before we talk about processing words, we need to decide what counts as a word. Let's start by looking at one particular **corpus** (plural **corpora**), a computer-readable collection of text or speech. For example the Brown corpus is a million-word collection of samples from 500 written English texts from different genres (newspaper, fiction, non-fiction, academic, etc.), assembled at Brown University in 1963–64 (Kučera and Francis, 1967). How many words are in the following Brown sentence?

He stepped out into the hall, was delighted to encounter a water brother.

This sentence has 13 words if we don't count punctuation marks as words, 15 if we count punctuation. Whether we treat period (“.”), comma (“,”), and so on as words depends on the task. Punctuation is critical for finding boundaries of things (commas, periods, colons) and for identifying some aspects of meaning (question marks, exclamation marks, quotation marks). For some tasks, like part-of-speech tagging or parsing or speech synthesis, we sometimes treat punctuation marks as if they were separate words.

The Switchboard corpus of American English telephone conversations between strangers was collected in the early 1990s; it contains 2430 conversations averaging 6 minutes each, totaling 240 hours of speech and about 3 million words (Godfrey et al., 1992). Such corpora of spoken language don't have punctuation but do introduce other complications with regard to defining words. Let's look at one utterance from Switchboard; an **utterance** is the spoken correlate of a sentence:

I do uh main- mainly business data processing

This utterance has two kinds of **disfluencies**. The broken-off word *main-* is called a **fragment**. Words like *uh* and *um* are called **fillers** or **filled pauses**. Should we consider these to be words? Again, it depends on the application. If we are building a speech transcription system, we might want to eventually strip out the disfluencies.

But we also sometimes keep disfluencies around. Disfluencies like *uh* or *um* are actually helpful in speech recognition in predicting the upcoming word, because they may signal that the speaker is restarting the clause or idea, and so for speech recognition they are treated as regular words. Because people use different disfluencies they can also be a cue to speaker identification. In fact Clark and Fox Tree (2002) showed that *uh* and *um* have different meanings. What do you think they are?

Are capitalized tokens like *They* and uncapitalized tokens like *they* the same word? These are lumped together in some tasks (speech recognition), while for part-of-speech or named-entity tagging, capitalization is a useful feature and is retained.

How about inflected forms like *cats* versus *cat*? These two words have the same **lemma** *cat* but are different wordforms. A **lemma** is a set of lexical forms having the same stem, the same major part-of-speech, and the same word sense. The **word-form** is the full inflected or derived form of the word. For morphologically complex languages like Arabic, we often need to deal with lemmatization. For many tasks in English, however, wordforms are sufficient.

How many words are there in English? To answer this question we need to distinguish two ways of talking about words. **Types** are the number of distinct words in a corpus; if the set of words in the vocabulary is V , the number of types is the vocabulary size $|V|$. **Tokens** are the total number N of running words. If we ignore punctuation, the following Brown sentence has 16 tokens and 14 types:

They picnicked by the pool, then lay back on the grass and looked at the stars.

When we speak about the number of words in the language, we are generally referring to word types.

Fig. 2.11 shows the rough numbers of types and tokens computed from some popular English corpora. The larger the corpora we look at, the more word types we find, and in fact this relationship between the number of types $|V|$ and number of tokens N is called **Herdan's Law** (Herdan, 1960) or **Heaps' Law** (Heaps, 1978) after its discoverers (in linguistics and information retrieval respectively). It is shown in Eq. 2.1, where k and β are positive constants, and $0 < \beta < 1$.

$$|V| = kN^\beta \quad (2.1)$$

Corpus	Tokens = N	Types = $ V $
Shakespeare	884 thousand	31 thousand
Brown corpus	1 million	38 thousand
Switchboard telephone conversations	2.4 million	20 thousand
COCA	440 million	2 million
Google n-grams	1 trillion	13 million

Figure 2.11 Rough numbers of types and tokens for some English language corpora. The largest, the Google n-grams corpus, contains 13 million types, but this count only includes types appearing 40 or more times, so the true number would be much larger.

The value of β depends on the corpus size and the genre, but at least for the large corpora in Fig. 2.11, β ranges from .67 to .75. Roughly then we can say that the vocabulary size for a text goes up significantly faster than the square root of its length in words.

Another measure of the number of words in the language is the number of lemmas instead of wordform types. Dictionaries can help in giving lemma counts; dictionary **entries** or **boldface forms** are a very rough upper bound on the number of lemmas (since some lemmas have multiple boldface forms). The 1989 edition of the Oxford English Dictionary had 615,000 entries.

2.3 Corpora

Words don't appear out of nowhere. Any particular piece of text that we study is produced by one or more specific speakers or writers, in a specific dialect of a specific language, at a specific time, in a specific place, for a specific function.

Perhaps the most important dimension of variation is the language. NLP algorithms are most useful when they apply across many languages. The world has 7097 languages at the time of this writing, according to the online Ethnologue catalog (Simons and Fennig, 2018). It is important to test algorithms on more than one language, and particularly on languages with different properties; by contrast there is an unfortunate current tendency for NLP algorithms to be developed or tested just on English (Bender, 2019). Even when algorithms are developed beyond English, they tend to be developed for the official languages of large industrialized nations (Chinese, Spanish, Japanese, German etc.), but we don't want to limit tools to just these few languages. Furthermore, most languages also have multiple varieties, often spoken in different regions or by different social groups. Thus, for example, if we're processing text that uses features of African American English (AAE) or African American Vernacular English (AAVE)—the variations of English used by millions of people in African American communities (King 2020)—we must use NLP tools that function with features of those varieties. Twitter posts might use features often used by speakers of African American English, such as constructions like *iont* (*I don't* in Mainstream American English (MAE)), or *talmbout* corresponding to MAE *talking about*, both examples that influence word segmentation (Blodgett et al. 2016, Jones 2015).

It's also quite common for speakers or writers to use multiple languages in a single communicative act, a phenomenon called **code switching**. Code switching is enormously common across the world; here are examples showing Spanish and (transliterated) Hindi code switching with English (Solorio et al. 2014, Jurgens et al. 2017):

- (2.2) Por primera vez veo a @username actually being hateful! it was beautiful:)
[For the first time I get to see @username actually being hateful! it was beautiful:]
- (2.3) dost tha or ra- hega ... dont worry ... but dherya rakhe
[“he was and will remain a friend ... don’t worry ... but have faith”]

Another dimension of variation is the genre. The text that our algorithms must process might come from newswire, fiction or non-fiction books, scientific articles, Wikipedia, or religious texts. It might come from spoken genres like telephone conversations, business meetings, police body-worn cameras, medical interviews, or transcripts of television shows or movies. It might come from work situations like doctors’ notes, legal text, or parliamentary or congressional proceedings.

Text also reflects the demographic characteristics of the writer (or speaker): their age, gender, race, socioeconomic class can all influence the linguistic properties of the text we are processing.

And finally, time matters too. Language changes over time, and for some languages we have good corpora of texts from different historical periods.

datasheet

Because language is so situated, when developing computational models for language processing from a corpus, it’s important to consider who produced the language, in what context, for what purpose. How can a user of a dataset know all these details? The best way is for the corpus creator to build a **datasheet** (Geburu et al., 2020) or **data statement** (Bender and Friedman, 2018) for each corpus. A datasheet specifies properties of a dataset like:

Motivation: Why was the corpus collected, by whom, and who funded it?

Situation: When and in what situation was the text written/spoken? For example, was there a task? Was the language originally spoken conversation, edited text, social media communication, monologue vs. dialogue?

Language variety: What language (including dialect/region) was the corpus in?

Speaker demographics: What was, e.g., the age or gender of the text’s authors?

Collection process: How big is the data? If it is a subsample how was it sampled? Was the data collected with consent? How was the data pre-processed, and what metadata is available?

Annotation process: What are the annotations, what are the demographics of the annotators, how were they trained, how was the data annotated?

Distribution: Are there copyright or other intellectual property restrictions?

2.4 Text Normalization

Before almost any natural language processing of a text, the text has to be normalized. At least three tasks are commonly applied as part of any normalization process:

1. Tokenizing (segmenting) words
2. Normalizing word formats
3. Segmenting sentences

In the next sections we walk through each of these tasks.

2.4.1 Unix Tools for Crude Tokenization and Normalization

Let's begin with an easy, if somewhat naive version of word tokenization and normalization (and frequency computation) that can be accomplished for English solely in a single UNIX command-line, inspired by Church (1994). We'll make use of some Unix commands: `tr`, used to systematically change particular characters in the input; `sort`, which sorts input lines in alphabetical order; and `uniq`, which collapses and counts adjacent identical lines.

For example let's begin with the 'complete words' of Shakespeare in one file, `sh.txt`. We can use `tr` to tokenize the words by changing every sequence of non-alphabetic characters to a newline ('A-Za-z' means alphabetic, the `-c` option complements to non-alphabet, and the `-s` option squeezes all sequences into a single character):

```
tr -sc 'A-Za-z' '\n' < sh.txt
```

The output of this command will be:

```
THE
SONNETS
by
William
Shakespeare
From
fairest
creatures
We
...
```

Now that there is one word per line, we can sort the lines, and pass them to `uniq -c` which will collapse and count them:

```
tr -sc 'A-Za-z' '\n' < sh.txt | sort | uniq -c
```

with the following output:

```
1945 A
72 AARON
19 ABBESS
25 Aaron
6 Abate
1 Abates
5 Abbess
6 Abbey
3 Abbot
...
```

Alternatively, we can collapse all the upper case to lower case:

```
tr -sc 'A-Za-z' '\n' < sh.txt | tr A-Z a-z | sort | uniq -c
```

whose output is

```
14725 a
97 aaron
1 abaissiez
10 abandon
2 abandoned
```

```

2 abase
1 abash
14 abate
3 abated
3 abatement
...

```

Now we can sort again to find the frequent words. The `-n` option to `sort` means to sort numerically rather than alphabetically, and the `-r` option means to sort in reverse order (highest-to-lowest):

```
tr -sc 'A-Za-z' '\n' < sh.txt | tr A-Z a-z | sort | uniq -c | sort -n -r
```

The results show that the most frequent words in Shakespeare, as in any other corpus, are the short **function words** like articles, pronouns, prepositions:

```

27378 the
26084 and
22538 i
19771 to
17481 of
14725 a
13826 you
12489 my
11318 that
11112 in
...

```

Unix tools of this sort can be very handy in building quick word count statistics for any corpus in English. While in some versions of Unix these command-line tools also correctly handle Unicode characters and so can be used for many languages, in general for handling most languages outside English we use more sophisticated tokenization algorithms.

2.4.2 Word Tokenization

tokenization

The simple UNIX tools above were fine for getting rough word statistics but more sophisticated algorithms are generally necessary for **tokenization**, the task of segmenting running text into words.

While the Unix command sequence just removed all the numbers and punctuation, for most NLP applications we'll need to keep these in our tokenization. We often want to break off punctuation as a separate token; commas are a useful piece of information for parsers, periods help indicate sentence boundaries. But we'll often want to keep the punctuation that occurs word internally, in examples like *m.p.h.*, *Ph.D.*, *AT&T*, and *cap'n*. Special characters and numbers will need to be kept in prices (\$45.55) and dates (01/02/06); we don't want to segment that price into separate tokens of "45" and "55". And there are URLs (<https://www.stanford.edu>), Twitter hashtags (#nlproc), or email addresses (someone@cs.colorado.edu).

Number expressions introduce other complications as well; while commas normally appear at word boundaries, commas are used inside numbers in English, every three digits: 555, 500.50. Languages, and hence tokenization requirements, differ on this; many continental European languages like Spanish, French, and German, by contrast, use a comma to mark the decimal point, and spaces (or sometimes periods) where English puts commas, for example, 555 500, 50.

clitic

A tokenizer can also be used to expand **clitic** contractions that are marked by apostrophes, for example, converting *what're* to the two tokens *what are*, and *we're* to *we are*. A clitic is a part of a word that can't stand on its own, and can only occur when it is attached to another word. Some such contractions occur in other alphabetic languages, including articles and pronouns in French (*j'ai*, *l'homme*).

Depending on the application, tokenization algorithms may also tokenize multiword expressions like *New York* or *rock 'n' roll* as a single token, which requires a multiword expression dictionary of some sort. Tokenization is thus intimately tied up with **named entity recognition**, the task of detecting names, dates, and organizations (Chapter 8).

Penn Treebank
tokenization

One commonly used tokenization standard is known as the **Penn Treebank tokenization** standard, used for the parsed corpora (treebanks) released by the Linguistic Data Consortium (LDC), the source of many useful datasets. This standard separates out clitics (*doesn't* becomes *does* plus *n't*), keeps hyphenated words together, and separates out all punctuation (to save space we're showing visible spaces '_' between tokens, although newlines is a more common output):

Input: "The San Francisco-based restaurant," they said,
"doesn't charge \$10".

Output: "_The_San_Francisco-based_restaurant_,_"_they_said_,_
"_does_n't_charge_\$10_"_.

In practice, since tokenization needs to be run before any other language processing, it needs to be very fast. The standard method for tokenization is therefore to use deterministic algorithms based on regular expressions compiled into very efficient finite state automata. For example, Fig. 2.12 shows an example of a basic regular expression that can be used to tokenize English with the `nltk.regexp_tokenize` function of the Python-based Natural Language Toolkit (NLTK) (Bird et al. 2009; <https://www.nltk.org>).

```
>>> text = 'That U.S.A. poster-print costs $12.40...'
>>> pattern = r'''(?x)      # set flag to allow verbose regexps
...     (?:[A-Z]\.)+        # abbreviations, e.g. U.S.A.
...     | \w+?:(-\w+)*       # words with optional internal hyphens
...     | \$?\d+(?:\.\d+)?%?  # currency, percentages, e.g. $12.40, 82%
...     | \.\.\.            # ellipsis
...     | [][.,;"'()?:_'-]  # these are separate tokens; includes ], [
... '''
>>> nltk.regexp_tokenize(text, pattern)
['That', 'U.S.A.', 'poster-print', 'costs', '$12.40', '...']
```

Figure 2.12 A Python trace of regular expression tokenization in the NLTK Python-based natural language processing toolkit (Bird et al., 2009), commented for readability; the `(?x)` verbose flag tells Python to strip comments and whitespace. Figure from Chapter 3 of Bird et al. (2009).

Carefully designed deterministic algorithms can deal with the ambiguities that arise, such as the fact that the apostrophe needs to be tokenized differently when used as a genitive marker (as in *the book's cover*), a quotative as in *'The other class', she said*, or in clitics like *they're*.

hanzi

Word tokenization is more complex in languages like written Chinese, Japanese, and Thai, which do not use spaces to mark potential word-boundaries. In Chinese, for example, words are composed of characters (called **hanzi** in Chinese). Each

character generally represents a single unit of meaning (called a **morpheme**) and is pronounceable as a single syllable. Words are about 2.4 characters long on average. But deciding what counts as a word in Chinese is complex. For example, consider the following sentence:

(2.4) 姚明进入总决赛
“Yao Ming reaches the finals”

As [Chen et al. \(2017b\)](#) point out, this could be treated as 3 words (‘Chinese Tree-bank’ segmentation):

(2.5) 姚明 进入 总决赛
YaoMing reaches finals

or as 5 words (‘Peking University’ segmentation):

(2.6) 姚 明 进 入 总 决 赛
Yao Ming reaches overall finals

Finally, it is possible in Chinese simply to ignore words altogether and use characters as the basic elements, treating the sentence as a series of 7 characters:

(2.7) 姚 明 进 入 总 决 赛
Yao Ming enter enter overall decision game

In fact, for most Chinese NLP tasks it turns out to work better to take characters rather than words as input, since characters are at a reasonable semantic level for most applications, and since most word standards, by contrast, result in a huge vocabulary with large numbers of very rare words ([Li et al., 2019b](#)).

word
segmentation

However, for Japanese and Thai the character is too small a unit, and so algorithms for **word segmentation** are required. These can also be useful for Chinese in the rare situations where word rather than character boundaries are required. The standard segmentation algorithms for these languages use neural **sequence models** trained via supervised machine learning on hand-segmented training sets; we’ll introduce sequence models in Chapter 8 and Chapter 9.

2.4.3 Byte-Pair Encoding for Tokenization

There is a third option to tokenizing text. Instead of defining tokens as words (whether delimited by spaces or more complex algorithms), or as characters (as in Chinese), we can use our data to automatically tell us what the tokens should be. This is especially useful in dealing with unknown words, an important problem in language processing. As we will see in the next chapter, NLP algorithms often learn some facts about language from one corpus (a **training** corpus) and then use these facts to make decisions about a separate **test** corpus and its language. Thus if our training corpus contains, say the words *low*, *new*, *newer*, but not *lower*, then if the word *lower* appears in our test corpus, our system will not know what to do with it.

subwords

To deal with this unknown word problem, modern tokenizers often automatically induce sets of tokens that include tokens smaller than words, called **subwords**. Subwords can be arbitrary substrings, or they can be meaning-bearing units like the morphemes *-est* or *-er*. (A morpheme is the smallest meaning-bearing unit of a language; for example the word *unlikeliest* has the morphemes *un-*, *likely*, and *-est*.) In modern tokenization schemes, most tokens are words, but some tokens are frequently occurring morphemes or other subwords like *-er*. Every unseen word like *lower* can thus be represented by some sequence of known subword units, such as *low* and *er*, or even as a sequence of individual letters if necessary.

Most tokenization schemes have two parts: a **token learner**, and a **token segmenter**. The token learner takes a raw training corpus (sometimes roughly pre-separated into words, for example by whitespace) and induces a vocabulary, a set of tokens. The token segmenter takes a raw test sentence and segments it into the tokens in the vocabulary. Three algorithms are widely used: **byte-pair encoding** (Sennrich et al., 2016), **unigram language modeling** (Kudo, 2018), and **WordPiece** (Schuster and Nakajima, 2012); there is also a **SentencePiece** library that includes implementations of the first two of the three (Kudo and Richardson, 2018a).

BPE In this section we introduce the simplest of the three, the **byte-pair encoding** or **BPE** algorithm (Sennrich et al., 2016); see Fig. 2.13. The BPE token learner begins with a vocabulary that is just the set of all individual characters. It then examines the training corpus, chooses the two symbols that are most frequently adjacent (say ‘A’, ‘B’), adds a new merged symbol ‘AB’ to the vocabulary, and replaces every adjacent ‘A’ ‘B’ in the corpus with the new ‘AB’. It continues to count and merge, creating new longer and longer character strings, until k merges have been done creating k novel tokens; k is thus a parameter of the algorithm. The resulting vocabulary consists of the original set of characters plus k new symbols.

The algorithm is usually run inside words (not merging across word boundaries), so the input corpus is first white-space-separated to give a set of strings, each corresponding to the characters of a word, plus a special end-of-word symbol `_`, and its counts. Let’s see its operation on the following tiny input corpus of 18 word tokens with counts for each word (the word *low* appears 5 times, the word *newer* 6 times, and so on), which would have a starting vocabulary of 11 letters:

corpus	vocabulary
5 l o w _	<code>_</code> , d, e, i, l, n, o, r, s, t, w
2 l o w e s t _	
6 n e w e r _	
3 w i d e r _	
2 n e w _	

The BPE algorithm first counts all pairs of adjacent symbols: the most frequent is the pair `e r` because it occurs in *newer* (frequency of 6) and *wider* (frequency of 3) for a total of 9 occurrences.¹ We then merge these symbols, treating `er` as one symbol, and count again:

corpus	vocabulary
5 l o w _	<code>_</code> , d, e, i, l, n, o, r, s, t, w, er
2 l o w e s t _	
6 n e w er _	
3 w i d er _	
2 n e w _	

Now the most frequent pair is `er _`, which we merge; our system has learned that there should be a token for word-final `er`, represented as `er_`:

corpus	vocabulary
5 l o w _	<code>_</code> , d, e, i, l, n, o, r, s, t, w, er, er_
2 l o w e s t _	
6 n e w er_	
3 w i d er_	
2 n e w _	

¹ Note that there can be ties; we could have instead chosen to merge `r _` first, since that also has a frequency of 9.

Next `n e` (total count of 8) get merged to `ne`:

corpus	vocabulary
5 l o w _	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne
2 l o w e s t _	
6 ne w er_	
3 w i d er_	
2 ne w _	

If we continue, the next merges are:

merge	current vocabulary
(ne, w)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new
(l, o)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo
(lo, w)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low
(new, er_)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low, newer_
(low, _)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low, newer_, low_

```

function BYTE-PAIR ENCODING(strings C, number of merges k) returns vocab V

  V ← all unique characters in C           # initial set of tokens is characters
  for i = 1 to k do                       # merge tokens k times
    tL, tR ← Most frequent pair of adjacent tokens in C
    tNEW ← tL + tR                       # make new token by concatenating
    V ← V + tNEW                           # update the vocabulary
    Replace each occurrence of tL, tR in C with tNEW # and update the corpus
  return V

```

Figure 2.13 The token learner part of the BPE algorithm for taking a corpus broken up into individual characters or bytes, and learning a vocabulary by iteratively merging tokens. Figure adapted from [Bostrom and Durrett \(2020\)](#).

Once we’ve learned our vocabulary, the token segmenter is used to tokenize a test sentence. The token segmenter just runs on the test data the merges we have learned from the training data, greedily, in the order we learned them. (Thus the frequencies in the test data don’t play a role, just the frequencies in the training data). So first we segment each test sentence word into characters. Then we apply the first rule: replace every instance of `e r` in the test corpus with `er`, and then the second rule: replace every instance of `er _` in the test corpus with `er_`, and so on. By the end, if the test corpus contained the character sequence `n e w e r _`, it would be tokenized as a full word. But the characters of a new (unknown) word like `l o w e r _` would be merged into the two tokens `low er_`.

Of course in real settings BPE is run with many thousands of merges on a very large input corpus. The result is that most words will be represented as full symbols, and only the very rare words (and unknown words) will have to be represented by their parts.

2.4.4 Word Normalization, Lemmatization and Stemming

normalization

Word **normalization** is the task of putting words/tokens in a standard format, choosing a single normal form for words with multiple forms like USA and US or uh-huh and uhhuh. This standardization may be valuable, despite the spelling information that is lost in the normalization process. For information retrieval or information

extraction about the US, we might want to see information from documents whether they mention the US or the USA.

case folding

Case folding is another kind of normalization. Mapping everything to lower case means that *Woodchuck* and *woodchuck* are represented identically, which is very helpful for generalization in many tasks, such as information retrieval or speech recognition. For sentiment analysis and other text classification tasks, information extraction, and machine translation, by contrast, case can be quite helpful and case folding is generally not done. This is because maintaining the difference between, for example, US the country and us the pronoun can outweigh the advantage in generalization that case folding would have provided for other words.

For many natural language processing situations we also want two morphologically different forms of a word to behave similarly. For example in web search, someone may type the string *woodchucks* but a useful system might want to also return pages that mention *woodchuck* with no *s*. This is especially common in morphologically complex languages like Polish, where for example the word *Warsaw* has different endings when it is the subject (*Warszawa*), or after a preposition like “in Warsaw” (*w Warszawie*), or “to Warsaw” (*do Warszawy*), and so on.

Lemmatization is the task of determining that two words have the same root, despite their surface differences. The words *am*, *are*, and *is* have the shared lemma *be*; the words *dinner* and *dinners* both have the lemma *dinner*. Lemmatizing each of these forms to the same lemma will let us find all mentions of words in Polish like *Warsaw*. The lemmatized form of a sentence like *He is reading detective stories* would thus be *He be read detective story*.

morpheme
stem
affix

How is lemmatization done? The most sophisticated methods for lemmatization involve complete **morphological parsing** of the word. **Morphology** is the study of the way words are built up from smaller meaning-bearing units called **morphemes**. Two broad classes of morphemes can be distinguished: **stems**—the central morpheme of the word, supplying the main meaning—and **affixes**—adding “additional” meanings of various kinds. So, for example, the word *fox* consists of one morpheme (the morpheme *fox*) and the word *cats* consists of two: the morpheme *cat* and the morpheme *-s*. A morphological parser takes a word like *cats* and parses it into the two morphemes *cat* and *s*, or parses a Spanish word like *amaren* (‘if in the future they would love’) into the morpheme *amar* ‘to love’, and the morphological features *3PL* and *future subjunctive*.

The Porter Stemmer

stemming
Porter stemmer

Lemmatization algorithms can be complex. For this reason we sometimes make use of a simpler but cruder method, which mainly consists of chopping off word-final affixes. This naive version of morphological analysis is called **stemming**. For example, the **Porter stemmer**, a widely used stemming algorithm (Porter, 1980), when applied to the following paragraph:

This was not the map we found in Billy Bones’s chest, but an accurate copy, complete in all things-names and heights and soundings-with the single exception of the red crosses and the written notes.

produces the following stemmed output:

Thi wa not the map we found in Billi Bone s chest but an accur copi complet in all thing name and height and sound with the singl except of the red cross and the written note

The algorithm is based on series of rewrite rules run in series: the output of each pass is fed as input to the next pass. Here are some sample rules (more details can be found at <https://tartarus.org/martin/PorterStemmer/>):

ATIONAL → ATE (e.g., relational → relate)
 ING → ϵ if the stem contains a vowel (e.g., motoring → motor)
 SSES → SS (e.g., grasses → grass)

Simple stemmers can be useful in cases where we need to collapse across different variants of the same lemma. Nonetheless, they do tend to commit errors of both over- and under-generalizing, as shown in the table below (Krovetz, 1993):

Errors of Commission		Errors of Omission	
organization	organ	European	Europe
doing	doe	analyzes	analysis
numerical	numerous	noisy	noise
policy	police	sparsity	sparse

2.4.5 Sentence Segmentation

sentence
segmentation

Sentence segmentation is another important step in text processing. The most useful cues for segmenting a text into sentences are punctuation, like periods, question marks, and exclamation points. Question marks and exclamation points are relatively unambiguous markers of sentence boundaries. Periods, on the other hand, are more ambiguous. The period character “.” is ambiguous between a sentence boundary marker and a marker of abbreviations like *Mr.* or *Inc.* The previous sentence that you just read showed an even more complex case of this ambiguity, in which the final period of *Inc.* marked both an abbreviation and the sentence boundary marker. For this reason, sentence tokenization and word tokenization may be addressed jointly.

In general, sentence tokenization methods work by first deciding (based on rules or machine learning) whether a period is part of the word or is a sentence-boundary marker. An abbreviation dictionary can help determine whether the period is part of a commonly used abbreviation; the dictionaries can be hand-built or machine-learned (Kiss and Strunk, 2006), as can the final sentence splitter. In the Stanford CoreNLP toolkit (Manning et al., 2014), for example sentence splitting is rule-based, a deterministic consequence of tokenization; a sentence ends when a sentence-ending punctuation (., !, or ?) is not already grouped with other characters into a token (such as for an abbreviation or number), optionally followed by additional final quotes or brackets.

2.5 Minimum Edit Distance

Much of natural language processing is concerned with measuring how similar two strings are. For example in spelling correction, the user typed some erroneous string—let’s say *graffe*—and we want to know what the user meant. The user probably intended a word that is similar to *graffe*. Among candidate similar words, the word *giraffe*, which differs by only one letter from *graffe*, seems intuitively to be more similar than, say *grail* or *graf*, which differ in more letters. Another

example comes from **coreference**, the task of deciding whether two strings such as the following refer to the same entity:

Stanford President Marc Tessier-Lavigne
Stanford University President Marc Tessier-Lavigne

Again, the fact that these two strings are very similar (differing by only one word) seems like useful evidence for deciding that they might be coreferent.

minimum edit
distance

Edit distance gives us a way to quantify both of these intuitions about string similarity. More formally, the **minimum edit distance** between two strings is defined as the minimum number of editing operations (operations like insertion, deletion, substitution) needed to transform one string into another.

alignment

The gap between *intention* and *execution*, for example, is 5 (delete an i, substitute e for n, substitute x for t, insert c, substitute u for n). It's much easier to see this by looking at the most important visualization for string distances, an **alignment** between the two strings, shown in Fig. 2.14. Given two sequences, an **alignment** is a correspondence between substrings of the two sequences. Thus, we say I **aligns** with the empty string, N with E, and so on. Beneath the aligned strings is another representation; a series of symbols expressing an **operation list** for converting the top string into the bottom string: **d** for deletion, **s** for substitution, **i** for insertion.

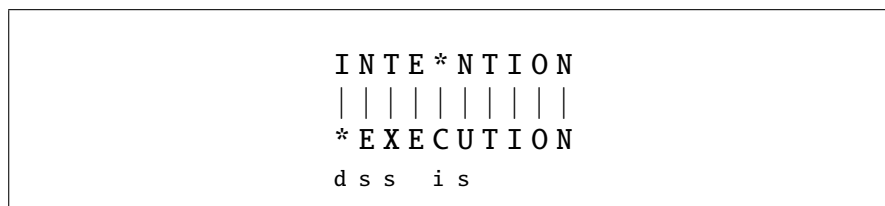


Figure 2.14 Representing the minimum edit distance between two strings as an **alignment**. The final row gives the operation list for converting the top string into the bottom string: d for deletion, s for substitution, i for insertion.

We can also assign a particular cost or weight to each of these operations. The **Levenshtein** distance between two sequences is the simplest weighting factor in which each of the three operations has a cost of 1 ([Levenshtein, 1966](#))—we assume that the substitution of a letter for itself, for example, t for t, has zero cost. The Levenshtein distance between *intention* and *execution* is 5. Levenshtein also proposed an alternative version of his metric in which each insertion or deletion has a cost of 1 and substitutions are not allowed. (This is equivalent to allowing substitution, but giving each substitution a cost of 2 since any substitution can be represented by one insertion and one deletion). Using this version, the Levenshtein distance between *intention* and *execution* is 8.

2.5.1 The Minimum Edit Distance Algorithm

How do we find the minimum edit distance? We can think of this as a search task, in which we are searching for the shortest path—a sequence of edits—from one string to another.

dynamic
programming

The space of all possible edits is enormous, so we can't search naively. However, lots of distinct edit paths will end up in the same state (string), so rather than recomputing all those paths, we could just remember the shortest path to a state each time we saw it. We can do this by using **dynamic programming**. Dynamic programming is the name for a class of algorithms, first introduced by [Bellman \(1957\)](#), that

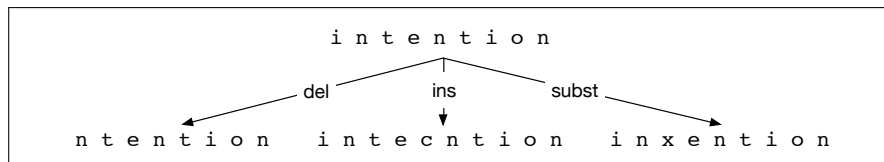


Figure 2.15 Finding the edit distance viewed as a search problem

apply a table-driven method to solve problems by combining solutions to subproblems. Some of the most commonly used algorithms in natural language processing make use of dynamic programming, such as the **Viterbi** algorithm (Chapter 8) and the **CKY** algorithm for parsing (Chapter 17).

The intuition of a dynamic programming problem is that a large problem can be solved by properly combining the solutions to various subproblems. Consider the shortest path of transformed words that represents the minimum edit distance between the strings *intention* and *execution* shown in Fig. 2.16.

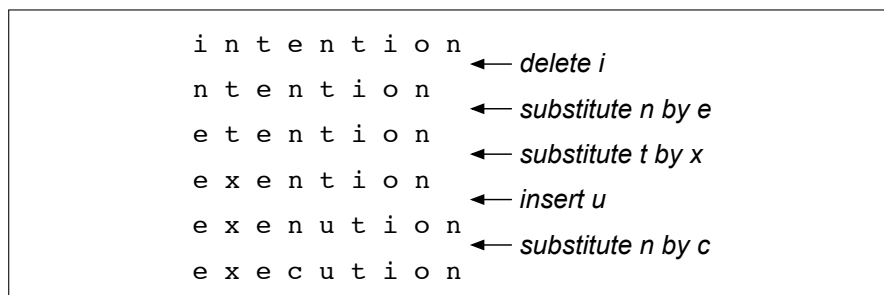


Figure 2.16 Path from *intention* to *execution*.

Imagine some string (perhaps it is *exention*) that is in this optimal path (whatever it is). The intuition of dynamic programming is that if *exention* is in the optimal operation list, then the optimal sequence must also include the optimal path from *intention* to *exention*. Why? If there were a shorter path from *intention* to *exention*, then we could use it instead, resulting in a shorter overall path, and the optimal sequence wouldn't be optimal, thus leading to a contradiction.

The **minimum edit distance algorithm** was named by [Wagner and Fischer \(1974\)](#) but independently discovered by many people (see the Historical Notes section of Chapter 8).

Let's first define the minimum edit distance between two strings. Given two strings, the source string X of length n , and target string Y of length m , we'll define $D[i, j]$ as the edit distance between $X[1..i]$ and $Y[1..j]$, i.e., the first i characters of X and the first j characters of Y . The edit distance between X and Y is thus $D[n, m]$.

We'll use dynamic programming to compute $D[n, m]$ bottom up, combining solutions to subproblems. In the base case, with a source substring of length i but an empty target string, going from i characters to 0 requires i deletes. With a target substring of length j but an empty source going from 0 characters to j characters requires j inserts. Having computed $D[i, j]$ for small i, j we then compute larger $D[i, j]$ based on previously computed smaller values. The value of $D[i, j]$ is computed by taking the minimum of the three possible paths through the matrix which arrive there:

$$D[i, j] = \min \begin{cases} D[i-1, j] + \text{del-cost}(\text{source}[i]) \\ D[i, j-1] + \text{ins-cost}(\text{target}[j]) \\ D[i-1, j-1] + \text{sub-cost}(\text{source}[i], \text{target}[j]) \end{cases} \quad (2.8)$$

minimum edit
distance
algorithm

If we assume the version of Levenshtein distance in which the insertions and deletions each have a cost of 1 ($\text{ins-cost}(\cdot) = \text{del-cost}(\cdot) = 1$), and substitutions have a cost of 2 (except substitution of identical letters have zero cost), the computation for $D[i, j]$ becomes:

$$D[i, j] = \min \begin{cases} D[i-1, j] + 1 \\ D[i, j-1] + 1 \\ D[i-1, j-1] + \begin{cases} 2; & \text{if } \text{source}[i] \neq \text{target}[j] \\ 0; & \text{if } \text{source}[i] = \text{target}[j] \end{cases} \end{cases} \quad (2.9)$$

The algorithm is summarized in Fig. 2.17; Fig. 2.18 shows the results of applying the algorithm to the distance between *intention* and *execution* with the version of Levenshtein in Eq. 2.9.

```

function MIN-EDIT-DISTANCE(source, target) returns min-distance

  n ← LENGTH(source)
  m ← LENGTH(target)
  Create a distance matrix  $D[n+1, m+1]$ 

  # Initialization: the zeroth row and column is the distance from the empty string
   $D[0, 0] = 0$ 
  for each row i from 1 to n do
     $D[i, 0] \leftarrow D[i-1, 0] + \text{del-cost}(\text{source}[i])$ 
  for each column j from 1 to m do
     $D[0, j] \leftarrow D[0, j-1] + \text{ins-cost}(\text{target}[j])$ 

  # Recurrence relation:
  for each row i from 1 to n do
    for each column j from 1 to m do
       $D[i, j] \leftarrow \text{MIN}( D[i-1, j] + \text{del-cost}(\text{source}[i]),$ 
                           $D[i-1, j-1] + \text{sub-cost}(\text{source}[i], \text{target}[j]),$ 
                           $D[i, j-1] + \text{ins-cost}(\text{target}[j]) )$ 

  # Termination
  return  $D[n, m]$ 

```

Figure 2.17 The minimum edit distance algorithm, an example of the class of dynamic programming algorithms. The various costs can either be fixed (e.g., $\forall x, \text{ins-cost}(x) = 1$) or can be specific to the letter (to model the fact that some letters are more likely to be inserted than others). We assume that there is no cost for substituting a letter for itself (i.e., $\text{sub-cost}(x, x) = 0$).

Alignment Knowing the minimum edit distance is useful for algorithms like finding potential spelling error corrections. But the edit distance algorithm is important in another way; with a small change, it can also provide the minimum cost **alignment** between two strings. Aligning two strings is useful throughout speech and language processing. In speech recognition, minimum edit distance alignment is used to compute the word error rate (Chapter 16). Alignment plays a role in machine translation, in which sentences in a parallel corpus (a corpus with a text in two languages) need to be matched to each other.

To extend the edit distance algorithm to produce an alignment, we can start by visualizing an alignment as a path through the edit distance matrix. Figure 2.19 shows this path with boldfaced cells. Each boldfaced cell represents an alignment

Src\Tar	#	e	x	e	c	u	t	i	o	n
#	0	1	2	3	4	5	6	7	8	9
i	1	2	3	4	5	6	7	6	7	8
n	2	3	4	5	6	7	8	7	8	7
t	3	4	5	6	7	8	7	8	9	8
e	4	3	4	5	6	7	8	9	10	9
n	5	4	5	6	7	8	9	10	11	10
t	6	5	6	7	8	9	8	9	10	11
i	7	6	7	8	9	10	9	8	9	10
o	8	7	8	9	10	11	10	9	8	9
n	9	8	9	10	11	12	11	10	9	8

Figure 2.18 Computation of minimum edit distance between *intention* and *execution* with the algorithm of Fig. 2.17, using Levenshtein distance with cost of 1 for insertions or deletions, 2 for substitutions.

of a pair of letters in the two strings. If two boldfaced cells occur in the same row, there will be an insertion in going from the source to the target; two boldfaced cells in the same column indicate a deletion.

Figure 2.19 also shows the intuition of how to compute this alignment path. The computation proceeds in two steps. In the first step, we augment the minimum edit distance algorithm to store backpointers in each cell. The backpointer from a cell points to the previous cell (or cells) that we came from in entering the current cell. We've shown a schematic of these backpointers in Fig. 2.19. Some cells have multiple backpointers because the minimum extension could have come from multiple previous cells. In the second step, we perform a **backtrace**. In a backtrace, we start from the last cell (at the final row and column), and follow the pointers back through the dynamic programming matrix. Each complete path between the final cell and the initial cell is a minimum distance alignment. Exercise 2.7 asks you to modify the minimum edit distance algorithm to store the pointers and compute the backtrace to output an alignment.

backtrace

	#	e	x	e	c	u	t	i	o	n
#	0	← 1	← 2	← 3	← 4	← 5	← 6	← 7	← 8	← 9
i	↑ 1	↖↗ 2	↖↗ 3	↖↗ 4	↖↗ 5	↖↗ 6	↖↗ 7	↖ 6	← 7	← 8
n	↑ 2	↖↗ 3	↖↗ 4	↖↗ 5	↖↗ 6	↖↗ 7	↖↗ 8	↑ 7	↖↗ 8	↖ 7
t	↑ 3	↖↗ 4	↖↗ 5	↖↗ 6	↖↗ 7	↖↗ 8	↖ 7	↖↗ 8	↖↗ 9	↑ 8
e	↑ 4	↖ 3	← 4	↖ 5	← 6	← 7	↖ 8	↖↗ 9	↖↗ 10	↑ 9
n	↑ 5	↑ 4	↖↗ 5	↖↗ 6	↖↗ 7	↖↗ 8	↖↗ 9	↖↗ 10	↖↗ 11	↖↗ 10
t	↑ 6	↑ 5	↖↗ 6	↖↗ 7	↖↗ 8	↖↗ 9	↖ 8	← 9	← 10	↖↗ 11
i	↑ 7	↑ 6	↖↗ 7	↖↗ 8	↖↗ 9	↖↗ 10	↑ 9	↖ 8	← 9	← 10
o	↑ 8	↑ 7	↖↗ 8	↖↗ 9	↖↗ 10	↖↗ 11	↑ 10	↑ 9	↖ 8	← 9
n	↑ 9	↑ 8	↖↗ 9	↖↗ 10	↖↗ 11	↖↗ 12	↑ 11	↑ 10	↑ 9	↖ 8

Figure 2.19 When entering a value in each cell, we mark which of the three neighboring cells we came from with up to three arrows. After the table is full we compute an **alignment** (minimum edit path) by using a **backtrace**, starting at the **8** in the lower-right corner and following the arrows back. The sequence of bold cells represents one possible minimum cost alignment between the two strings. Diagram design after Gusfield (1997).

While we worked our example with simple Levenshtein distance, the algorithm in Fig. 2.17 allows arbitrary weights on the operations. For spelling correction, for example, substitutions are more likely to happen between letters that are next to

each other on the keyboard. The **Viterbi** algorithm is a probabilistic extension of minimum edit distance. Instead of computing the “minimum edit distance” between two strings, Viterbi computes the “maximum probability alignment” of one string with another. We’ll discuss this more in Chapter 8.

2.6 Summary

This chapter introduced a fundamental tool in language processing, the **regular expression**, and showed how to perform basic **text normalization** tasks including **word segmentation** and **normalization**, **sentence segmentation**, and **stemming**. We also introduced the important **minimum edit distance** algorithm for comparing strings. Here’s a summary of the main points we covered about these ideas:

- The **regular expression** language is a powerful tool for pattern-matching.
- Basic operations in regular expressions include **concatenation** of symbols, **disjunction** of symbols (`[]`, `|`, and `.`), **counters** (`*`, `+`, and `{n,m}`), **anchors** (`^`, `$`) and precedence operators (`(,)`).
- **Word tokenization and normalization** are generally done by cascades of simple regular expression substitutions or finite automata.
- The **Porter algorithm** is a simple and efficient way to do **stemming**, stripping off affixes. It does not have high accuracy but may be useful for some tasks.
- The **minimum edit distance** between two strings is the minimum number of operations it takes to edit one into the other. Minimum edit distance can be computed by **dynamic programming**, which also results in an **alignment** of the two strings.

Bibliographical and Historical Notes

Kleene 1951; 1956 first defined regular expressions and the finite automaton, based on the McCulloch-Pitts neuron. Ken Thompson was one of the first to build regular expressions compilers into editors for text searching (Thompson, 1968). His editor *ed* included a command “g/regular expression/p”, or Global Regular Expression Print, which later became the Unix *grep* utility.

Text normalization algorithms have been applied since the beginning of the field. One of the earliest widely used stemmers was Lovins (1968). Stemming was also applied early to the digital humanities, by Packard (1973), who built an affix-stripping morphological parser for Ancient Greek. Currently a wide variety of code for tokenization and normalization is available, such as the Stanford Tokenizer (<https://nlp.stanford.edu/software/tokenizer.shtml>) or specialized tokenizers for Twitter (O’Connor et al., 2010), or for sentiment (<http://sentiment.christopherpotts.net/tokenizing.html>). See Palmer (2012) for a survey of text preprocessing. NLTK is an essential tool that offers both useful Python libraries (<https://www.nltk.org>) and textbook descriptions (Bird et al., 2009) of many algorithms including text normalization and corpus interfaces.

For more on Herdan’s law and Heaps’ Law, see Herdan (1960, p. 28), Heaps (1978), Egghe (2007) and Baayen (2001); Yasseri et al. (2012) discuss the relationship with other measures of linguistic complexity. For more on edit distance, see the excellent Gusfield (1997). Our example measuring the edit distance from ‘intention’

CHAPTER

7

Neural Networks and Neural Language Models

“[M]achines of this character can behave in a very complicated manner when the number of units is large.”

Alan Turing (1948) “Intelligent Machines”, page 6

Neural networks are a fundamental computational tool for language processing, and a very old one. They are called neural because their origins lie in the **McCulloch-Pitts neuron** (McCulloch and Pitts, 1943), a simplified model of the human neuron as a kind of computing element that could be described in terms of propositional logic. But the modern use in language processing no longer draws on these early biological inspirations.

feedforward

deep learning

Instead, a modern neural network is a network of small computing units, each of which takes a vector of input values and produces a single output value. In this chapter we introduce the neural net applied to classification. The architecture we introduce is called a **feedforward network** because the computation proceeds iteratively from one layer of units to the next. The use of modern neural nets is often called **deep learning**, because modern networks are often **deep** (have many layers).

Neural networks share much of the same mathematics as logistic regression. But neural networks are a more powerful classifier than logistic regression, and indeed a minimal neural network (technically one with a single ‘hidden layer’) can be shown to learn any function.

Neural net classifiers are different from logistic regression in another way. With logistic regression, we applied the regression classifier to many different tasks by developing many rich kinds of feature templates based on domain knowledge. When working with neural networks, it is more common to avoid most uses of rich hand-derived features, instead building neural networks that take raw words as inputs and learn to induce features as part of the process of learning to classify. We saw examples of this kind of representation learning for embeddings in Chapter 6. Nets that are very deep are particularly good at representation learning. For that reason deep neural nets are the right tool for tasks that offer sufficient data to learn features automatically.

In this chapter we’ll introduce feedforward networks as classifiers, and also apply them to the simple task of language modeling: assigning probabilities to word sequences and predicting upcoming words. In subsequent chapters we’ll introduce many other aspects of neural models, such as **recurrent neural networks** (Chapter 9), the **Transformer** (Chapter 10), and masked language modeling (Chapter 11).

7.1 Units

The building block of a neural network is a single computational unit. A unit takes a set of real valued numbers as input, performs some computation on them, and produces an output.

At its heart, a neural unit is taking a weighted sum of its inputs, with one additional term in the sum called a **bias term**. Given a set of inputs $x_1 \dots x_n$, a unit has a set of corresponding weights $w_1 \dots w_n$ and a bias b , so the weighted sum z can be represented as:

$$z = b + \sum_i w_i x_i \quad (7.1)$$

Often it's more convenient to express this weighted sum using vector notation; recall from linear algebra that a **vector** is, at heart, just a list or array of numbers. Thus we'll talk about z in terms of a weight vector $\mathbf{w}$, a scalar bias b , and an input vector $\mathbf{x}$, and we'll replace the sum with the convenient **dot product**:

$$z = \mathbf{w} \cdot \mathbf{x} + b \quad (7.2)$$

As defined in Eq. 7.2, z is just a real valued number.

Finally, instead of using z , a linear function of x , as the output, neural units apply a non-linear function f to z . We will refer to the output of this function as the **activation** value for the unit, a . Since we are just modeling a single unit, the activation for the node is in fact the final output of the network, which we'll generally call y . So the value y is defined as:

$$y = a = f(z)$$

We'll discuss three popular non-linear functions $f()$ below (the sigmoid, the tanh, and the rectified linear unit or ReLU) but it's pedagogically convenient to start with the **sigmoid** function since we saw it in Chapter 5:

$$y = \sigma(z) = \frac{1}{1 + e^{-z}} \quad (7.3)$$

The sigmoid (shown in Fig. 7.1) has a number of advantages; it maps the output into the range $(0, 1)$, which is useful in squashing outliers toward 0 or 1. And it's differentiable, which as we saw in Section 5.10 will be handy for learning.

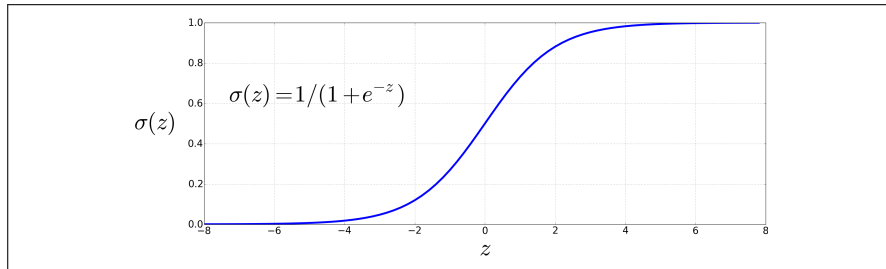


Figure 7.1 The sigmoid function takes a real value and maps it to the range $(0, 1)$. It is nearly linear around 0 but outlier values get squashed toward 0 or 1.

Substituting Eq. 7.2 into Eq. 7.3 gives us the output of a neural unit:

$$y = \sigma(\mathbf{w} \cdot \mathbf{x} + b) = \frac{1}{1 + \exp(-(\mathbf{w} \cdot \mathbf{x} + b))} \quad (7.4)$$

Fig. 7.2 shows a final schematic of a basic neural unit. In this example the unit takes 3 input values x_1, x_2 , and x_3 , and computes a weighted sum, multiplying each value by a weight (w_1, w_2 , and w_3 , respectively), adds them to a bias term b , and then passes the resulting sum through a sigmoid function to result in a number between 0 and 1.

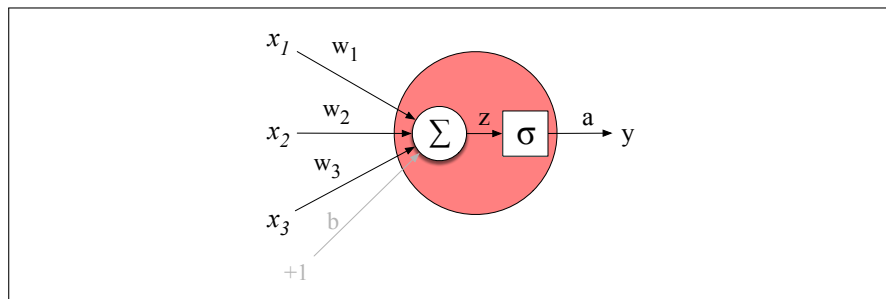


Figure 7.2 A neural unit, taking 3 inputs x_1, x_2 , and x_3 (and a bias b that we represent as a weight for an input clamped at +1) and producing an output y . We include some convenient intermediate variables: the output of the summation, z , and the output of the sigmoid, a . In this case the output of the unit y is the same as a , but in deeper networks we'll reserve y to mean the final output of the entire network, leaving a as the activation of an individual node.

Let's walk through an example just to get an intuition. Let's suppose we have a unit with the following weight vector and bias:

$$\begin{aligned}\mathbf{w} &= [0.2, 0.3, 0.9] \\ b &= 0.5\end{aligned}$$

What would this unit do with the following input vector:

$$\mathbf{x} = [0.5, 0.6, 0.1]$$

The resulting output y would be:

$$y = \sigma(\mathbf{w} \cdot \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w} \cdot \mathbf{x} + b)}} = \frac{1}{1 + e^{-(.5 \cdot .2 + .6 \cdot .3 + .1 \cdot .9 + .5)}} = \frac{1}{1 + e^{-0.87}} = .70$$

tanh In practice, the sigmoid is not commonly used as an activation function. A function that is very similar but almost always better is the **tanh** function shown in Fig. 7.3a; tanh is a variant of the sigmoid that ranges from -1 to +1:

$$y = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \quad (7.5)$$

ReLU The simplest activation function, and perhaps the most commonly used, is the rectified linear unit, also called the **ReLU**, shown in Fig. 7.3b. It's just the same as z when z is positive, and 0 otherwise:

$$y = \text{ReLU}(z) = \max(z, 0) \quad (7.6)$$

These activation functions have different properties that make them useful for different language applications or network architectures. For example, the tanh function has the nice properties of being smoothly differentiable and mapping outlier values toward the mean. The rectifier function, on the other hand, has nice properties that

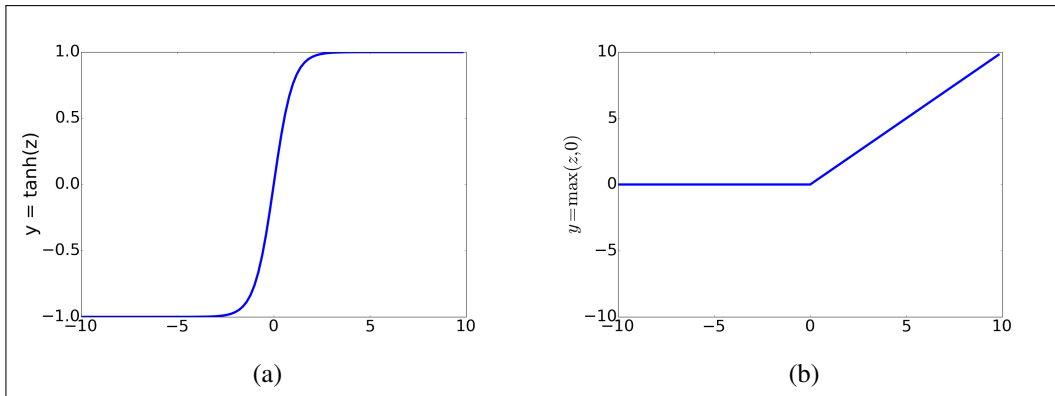


Figure 7.3 The tanh and ReLU activation functions.

result from it being very close to linear. In the sigmoid or tanh functions, very high values of z result in values of y that are **saturated**, i.e., extremely close to 1, and have derivatives very close to 0. Zero derivatives cause problems for learning, because as we'll see in Section 7.6, we'll train networks by propagating an error signal backwards, multiplying gradients (partial derivatives) from each layer of the network; gradients that are almost 0 cause the error signal to get smaller and smaller until it is too small to be used for training, a problem called the **vanishing gradient** problem. Rectifiers don't have this problem, since the derivative of ReLU for high values of z is 1 rather than very close to 0.

7.2 The XOR problem

Early in the history of neural networks it was realized that the power of neural networks, as with the real neurons that inspired them, comes from combining these units into larger networks.

One of the most clever demonstrations of the need for multi-layer networks was the proof by [Minsky and Papert \(1969\)](#) that a single neural unit cannot compute some very simple functions of its input. Consider the task of computing elementary logical functions of two inputs, like AND, OR, and XOR. As a reminder, here are the truth tables for those functions:

AND			OR			XOR		
x1	x2	y	x1	x2	y	x1	x2	y
0	0	0	0	0	0	0	0	0
0	1	0	0	1	1	0	1	1
1	0	0	1	0	1	1	0	1
1	1	1	1	1	1	1	1	0

This example was first shown for the **perceptron**, which is a very simple neural unit that has a binary output and does **not** have a non-linear activation function. The output y of a perceptron is 0 or 1, and is computed as follows (using the same weight $\mathbf{w}$, input $\mathbf{x}$, and bias b as in Eq. 7.2):

$$y = \begin{cases} 0, & \text{if } \mathbf{w} \cdot \mathbf{x} + b \leq 0 \\ 1, & \text{if } \mathbf{w} \cdot \mathbf{x} + b > 0 \end{cases} \quad (7.7)$$

It's very easy to build a perceptron that can compute the logical AND and OR functions of its binary inputs; Fig. 7.4 shows the necessary weights.

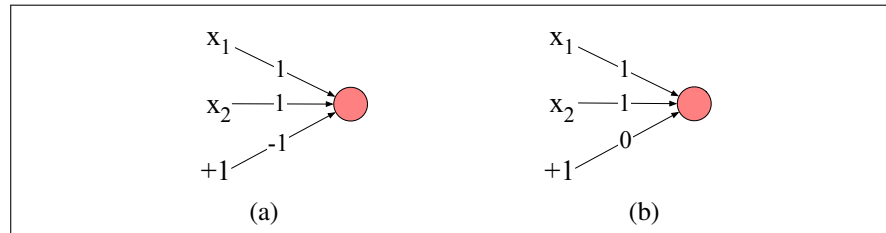


Figure 7.4 The weights w and bias b for perceptrons for computing logical functions. The inputs are shown as x_1 and x_2 and the bias as a special node with value $+1$ which is multiplied with the bias weight b . (a) logical AND, with weights $w_1 = 1$ and $w_2 = 1$ and bias weight $b = -1$. (b) logical OR, with weights $w_1 = 1$ and $w_2 = 1$ and bias weight $b = 0$. These weights/biases are just one from an infinite number of possible sets of weights and biases that would implement the functions.

It turns out, however, that it's not possible to build a perceptron to compute logical XOR! (It's worth spending a moment to give it a try!)

The intuition behind this important result relies on understanding that a perceptron is a linear classifier. For a two-dimensional input x_1 and x_2 , the perceptron equation, $w_1x_1 + w_2x_2 + b = 0$ is the equation of a line. (We can see this by putting it in the standard linear format: $x_2 = (-w_1/w_2)x_1 + (-b/w_2)$.) This line acts as a **decision boundary** in two-dimensional space in which the output 0 is assigned to all inputs lying on one side of the line, and the output 1 to all input points lying on the other side of the line. If we had more than 2 inputs, the decision boundary becomes a hyperplane instead of a line, but the idea is the same, separating the space into two categories.

Fig. 7.5 shows the possible logical inputs (00, 01, 10, and 11) and the line drawn by one possible set of parameters for an AND and an OR classifier. Notice that there is simply no way to draw a line that separates the positive cases of XOR (01 and 10) from the negative cases (00 and 11). We say that XOR is not a **linearly separable** function. Of course we could draw a boundary with a curve, or some other function, but not a single line.

7.2.1 The solution: neural networks

While the XOR function cannot be calculated by a single perceptron, it can be calculated by a layered network of perceptron units. Rather than see this with networks of simple perceptrons, however, let's see how to compute XOR using two layers of ReLU-based units following Goodfellow et al. (2016). Fig. 7.6 shows a figure with the input being processed by two layers of neural units. The middle layer (called h) has two units, and the output layer (called y) has one unit. A set of weights and biases are shown for each ReLU that correctly computes the XOR function.

Let's walk through what happens with the input $\mathbf{x} = [0, 0]$. If we multiply each input value by the appropriate weight, sum, and then add the bias b , we get the vector $[0, -1]$, and we then apply the rectified linear transformation to give the output of the h layer as $[0, 0]$. Now we once again multiply by the weights, sum, and add the bias (0 in this case) resulting in the value 0. The reader should work through the computation of the remaining 3 possible input pairs to see that the resulting y values are 1 for the inputs $[0, 1]$ and $[1, 0]$ and 0 for $[0, 0]$ and $[1, 1]$.

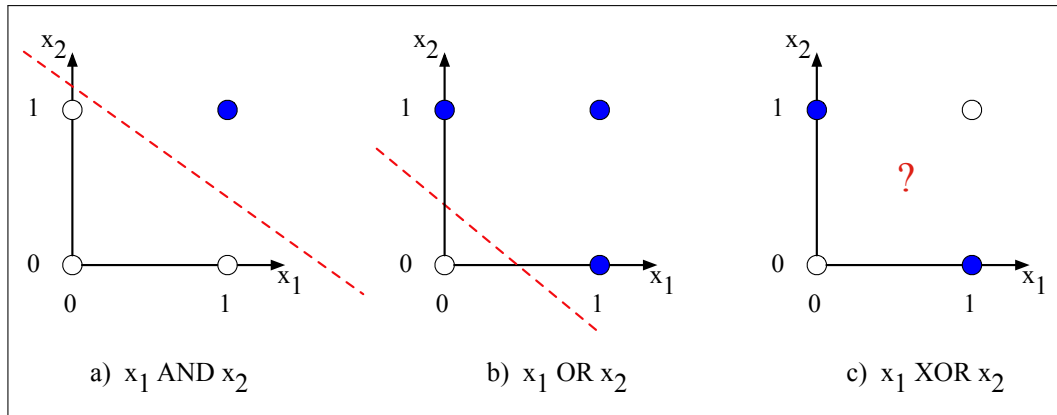


Figure 7.5 The functions AND, OR, and XOR, represented with input x_1 on the x-axis and input x_2 on the y-axis. Filled circles represent perceptron outputs of 1, and white circles perceptron outputs of 0. There is no way to draw a line that correctly separates the two categories for XOR. Figure styled after Russell and Norvig (2002).

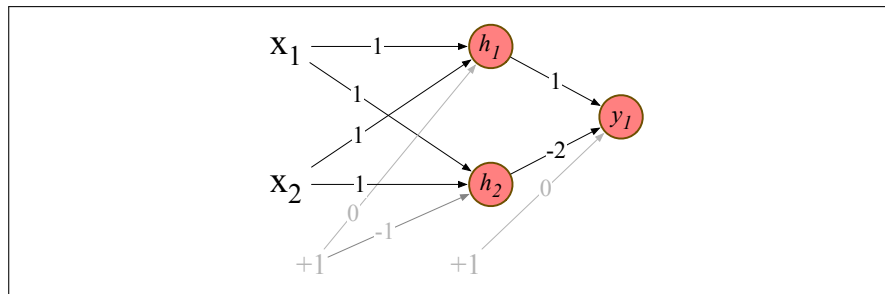


Figure 7.6 XOR solution after Goodfellow et al. (2016). There are three ReLU units, in two layers; we've called them h_1 , h_2 (h for “hidden layer”) and y_1 . As before, the numbers on the arrows represent the weights w for each unit, and we represent the bias b as a weight on a unit clamped to +1, with the bias weights/units in gray.

It's also instructive to look at the intermediate results, the outputs of the two hidden nodes h_1 and h_2 . We showed in the previous paragraph that the $\mathbf{h}$ vector for the inputs $\mathbf{x} = [0, 0]$ was $[0, 0]$. Fig. 7.7b shows the values of the $\mathbf{h}$ layer for all 4 inputs. Notice that hidden representations of the two input points $\mathbf{x} = [0, 1]$ and $\mathbf{x} = [1, 0]$ (the two cases with XOR output = 1) are merged to the single point $\mathbf{h} = [1, 0]$. The merger makes it easy to linearly separate the positive and negative cases of XOR. In other words, we can view the hidden layer of the network as forming a representation of the input.

In this example we just stipulated the weights in Fig. 7.6. But for real examples the weights for neural networks are learned automatically using the error backpropagation algorithm to be introduced in Section 7.6. That means the hidden layers will learn to form useful representations. This intuition, that neural networks can automatically learn useful representations of the input, is one of their key advantages, and one that we will return to again and again in later chapters.

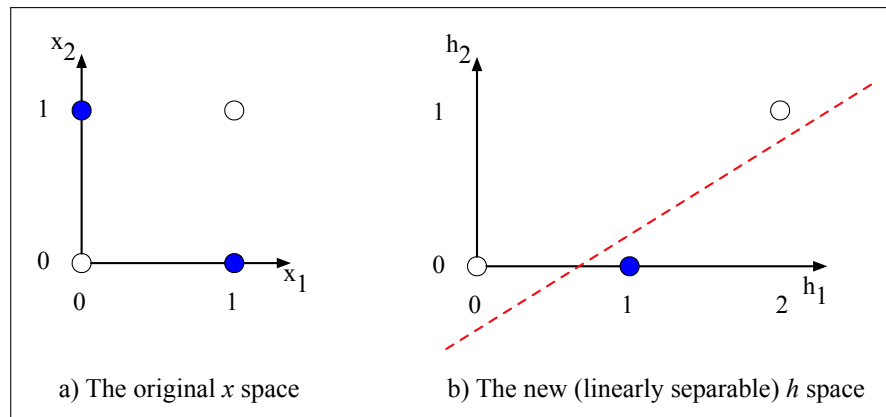


Figure 7.7 The hidden layer forming a new representation of the input. (b) shows the representation of the hidden layer, $\mathbf{h}$, compared to the original input representation $\mathbf{x}$ in (a). Notice that the input point $[0, 1]$ has been collapsed with the input point $[1, 0]$, making it possible to linearly separate the positive and negative cases of XOR. After Goodfellow et al. (2016).

7.3 Feedforward Neural Networks

feedforward
network

Let's now walk through a slightly more formal presentation of the simplest kind of neural network, the **feedforward network**. A feedforward network is a multilayer network in which the units are connected with no cycles; the outputs from units in each layer are passed to units in the next higher layer, and no outputs are passed back to lower layers. (In Chapter 9 we'll introduce networks with cycles, called **recurrent neural networks**.)

multi-layer
perceptrons
MLP

For historical reasons multilayer networks, especially feedforward networks, are sometimes called **multi-layer perceptrons** (or **MLPs**); this is a technical misnomer, since the units in modern multilayer networks aren't perceptrons (perceptrons are purely linear, but modern networks are made up of units with non-linearities like sigmoids), but at some point the name stuck.

Simple feedforward networks have three kinds of nodes: input units, hidden units, and output units.

Fig. 7.8 shows a picture. The input layer $\mathbf{x}$ is a vector of simple scalar values just as we saw in Fig. 7.2.

hidden layer

fully-connected

The core of the neural network is the **hidden layer** $\mathbf{h}$ formed of **hidden units** h_i , each of which is a neural unit as described in Section 7.1, taking a weighted sum of its inputs and then applying a non-linearity. In the standard architecture, each layer is **fully-connected**, meaning that each unit in each layer takes as input the outputs from all the units in the previous layer, and there is a link between every pair of units from two adjacent layers. Thus each hidden unit sums over all the input units.

Recall that a single hidden unit has as parameters a weight vector and a bias. We represent the parameters for the entire hidden layer by combining the weight vector and bias for each unit i into a single weight matrix $\mathbf{W}$ and a single bias vector $\mathbf{b}$ for the whole layer (see Fig. 7.8). Each element $\mathbf{W}_{ji}$ of the weight matrix $\mathbf{W}$ represents the weight of the connection from the i th input unit x_i to the j th hidden unit h_j .

The advantage of using a single matrix $\mathbf{W}$ for the weights of the entire layer is that now the hidden layer computation for a feedforward network can be done very efficiently with simple matrix operations. In fact, the computation only has three

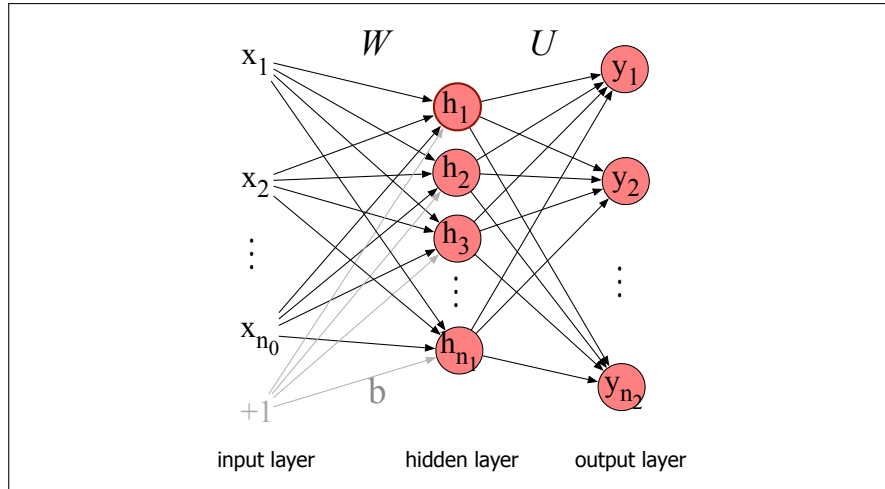


Figure 7.8 A simple 2-layer feedforward network, with one hidden layer, one output layer, and one input layer (the input layer is usually not counted when enumerating layers).

steps: multiplying the weight matrix by the input vector $\mathbf{x}$, adding the bias vector $\mathbf{b}$, and applying the activation function g (such as the sigmoid, tanh, or ReLU activation function defined above).

The output of the hidden layer, the vector $\mathbf{h}$, is thus the following (for this example we'll use the sigmoid function σ as our activation function):

$$\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b}) \quad (7.8)$$

Notice that we're applying the σ function here to a vector, while in Eq. 7.3 it was applied to a scalar. We're thus allowing $\sigma(\cdot)$, and indeed any activation function $g(\cdot)$, to apply to a vector element-wise, so $g[z_1, z_2, z_3] = [g(z_1), g(z_2), g(z_3)]$.

Let's introduce some constants to represent the dimensionalities of these vectors and matrices. We'll refer to the input layer as layer 0 of the network, and have n_0 represent the number of inputs, so $\mathbf{x}$ is a vector of real numbers of dimension n_0 , or more formally $\mathbf{x} \in \mathbb{R}^{n_0}$, a column vector of dimensionality $[n_0, 1]$. Let's call the hidden layer layer 1 and the output layer layer 2. The hidden layer has dimensionality n_1 , so $\mathbf{h} \in \mathbb{R}^{n_1}$ and also $\mathbf{b} \in \mathbb{R}^{n_1}$ (since each hidden unit can take a different bias value). And the weight matrix $\mathbf{W}$ has dimensionality $\mathbf{W} \in \mathbb{R}^{n_1 \times n_0}$, i.e. $[n_1, n_0]$.

Take a moment to convince yourself that the matrix multiplication in Eq. 7.8 will compute the value of each $\mathbf{h}_j$ as $\sigma(\sum_{i=1}^{n_0} \mathbf{W}_{ji}\mathbf{x}_i + \mathbf{b}_j)$.

As we saw in Section 7.2, the resulting value $\mathbf{h}$ (for *hidden* but also for *hypothesis*) forms a *representation* of the input. The role of the output layer is to take this new representation $\mathbf{h}$ and compute a final output. This output could be a real-valued number, but in many cases the goal of the network is to make some sort of classification decision, and so we will focus on the case of classification.

If we are doing a binary task like sentiment classification, we might have a single output node, and its scalar value y is the probability of positive versus negative sentiment. If we are doing multinomial classification, such as assigning a part-of-speech tag, we might have one output node for each potential part-of-speech, whose output value is the probability of that part-of-speech, and the values of all the output nodes must sum to one. The output layer is thus a vector $\mathbf{y}$ that gives a probability distribution across the output nodes.

Let's see how this happens. Like the hidden layer, the output layer has a weight matrix (let's call it $\mathbf{U}$), but some models don't include a bias vector $\mathbf{b}$ in the output

layer, so we'll simplify by eliminating the bias vector in this example. The weight matrix is multiplied by its input vector ($\mathbf{h}$) to produce the intermediate output $\mathbf{z}$:

$$\mathbf{z} = \mathbf{U}\mathbf{h}$$

There are n_2 output nodes, so $\mathbf{z} \in \mathbb{R}^{n_2}$, weight matrix $\mathbf{U}$ has dimensionality $\mathbf{U} \in \mathbb{R}^{n_2 \times n_1}$, and element $\mathbf{U}_{ij}$ is the weight from unit j in the hidden layer to unit i in the output layer.

However, $\mathbf{z}$ can't be the output of the classifier, since it's a vector of real-valued numbers, while what we need for classification is a vector of probabilities. There is a convenient function for **normalizing** a vector of real values, by which we mean converting it to a vector that encodes a probability distribution (all the numbers lie between 0 and 1 and sum to 1): the **softmax** function that we saw on page 87 of Chapter 5. More generally for any vector $\mathbf{z}$ of dimensionality d , the softmax is defined as:

$$\text{softmax}(\mathbf{z}_i) = \frac{\exp(\mathbf{z}_i)}{\sum_{j=1}^d \exp(\mathbf{z}_j)} \quad 1 \leq i \leq d \quad (7.9)$$

Thus for example given a vector

$$\mathbf{z} = [0.6, 1.1, -1.5, 1.2, 3.2, -1.1], \quad (7.10)$$

the softmax function will normalize it to a probability distribution (shown rounded):

$$\text{softmax}(\mathbf{z}) = [0.055, 0.090, 0.0067, 0.10, 0.74, 0.010] \quad (7.11)$$

You may recall that we used softmax to create a probability distribution from a vector of real-valued numbers (computed from summing weights times features) in the multinomial version of logistic regression in Chapter 5.

That means we can think of a neural network classifier with one hidden layer as building a vector $\mathbf{h}$ which is a hidden layer representation of the input, and then running standard multinomial logistic regression on the features that the network develops in $\mathbf{h}$. By contrast, in Chapter 5 the features were mainly designed by hand via feature templates. So a neural network is like multinomial logistic regression, but (a) with many layers, since a deep neural network is like layer after layer of logistic regression classifiers; (b) with those intermediate layers having many possible activation functions (tanh, ReLU, sigmoid) instead of just sigmoid (although we'll continue to use σ for convenience to mean any activation function); (c) rather than forming the features by feature templates, the prior layers of the network induce the feature representations themselves.

Here are the final equations for a feedforward network with a single hidden layer, which takes an input vector $\mathbf{x}$, outputs a probability distribution $\mathbf{y}$, and is parameterized by weight matrices $\mathbf{W}$ and $\mathbf{U}$ and a bias vector $\mathbf{b}$:

$$\begin{aligned} \mathbf{h} &= \sigma(\mathbf{W}\mathbf{x} + \mathbf{b}) \\ \mathbf{z} &= \mathbf{U}\mathbf{h} \\ \mathbf{y} &= \text{softmax}(\mathbf{z}) \end{aligned} \quad (7.12)$$

And just to remember the shapes of all our variables, $\mathbf{x} \in \mathbb{R}^{n_0}$, $\mathbf{h} \in \mathbb{R}^{n_1}$, $\mathbf{b} \in \mathbb{R}^{n_1}$, $\mathbf{W} \in \mathbb{R}^{n_1 \times n_0}$, $\mathbf{U} \in \mathbb{R}^{n_2 \times n_1}$, and the output vector $\mathbf{y} \in \mathbb{R}^{n_2}$. We'll call this network a 2-layer network (we traditionally don't count the input layer when numbering layers, but do count the output layer). So by this terminology logistic regression is a 1-layer network.

7.3.1 More details on feedforward networks

Let's now set up some notation to make it easier to talk about deeper networks of depth more than 2. We'll use superscripts in square brackets to mean layer numbers, starting at 0 for the input layer. So $\mathbf{W}^{[1]}$ will mean the weight matrix for the (first) hidden layer, and $\mathbf{b}^{[1]}$ will mean the bias vector for the (first) hidden layer. n_j will mean the number of units at layer j . We'll use $g(\cdot)$ to stand for the activation function, which will tend to be ReLU or tanh for intermediate layers and softmax for output layers. We'll use $\mathbf{a}^{[i]}$ to mean the output from layer i , and $\mathbf{z}^{[i]}$ to mean the combination of weights and biases $\mathbf{W}^{[i]}\mathbf{a}^{[i-1]} + \mathbf{b}^{[i]}$. The 0th layer is for inputs, so we'll refer to the inputs $\mathbf{x}$ more generally as $\mathbf{a}^{[0]}$.

Thus we can re-represent our 2-layer net from Eq. 7.12 as follows:

$$\begin{aligned}\mathbf{z}^{[1]} &= \mathbf{W}^{[1]}\mathbf{a}^{[0]} + \mathbf{b}^{[1]} \\ \mathbf{a}^{[1]} &= g^{[1]}(\mathbf{z}^{[1]}) \\ \mathbf{z}^{[2]} &= \mathbf{W}^{[2]}\mathbf{a}^{[1]} + \mathbf{b}^{[2]} \\ \mathbf{a}^{[2]} &= g^{[2]}(\mathbf{z}^{[2]}) \\ \hat{\mathbf{y}} &= \mathbf{a}^{[2]}\end{aligned}\tag{7.13}$$

Note that with this notation, the equations for the computation done at each layer are the same. The algorithm for computing the forward step in an n -layer feedforward network, given the input vector $\mathbf{a}^{[0]}$ is thus simply:

```
for  $i$  in 1,...,n
   $\mathbf{z}^{[i]} = \mathbf{W}^{[i]}\mathbf{a}^{[i-1]} + \mathbf{b}^{[i]}$ 
   $\mathbf{a}^{[i]} = g^{[i]}(\mathbf{z}^{[i]})$ 
 $\hat{\mathbf{y}} = \mathbf{a}^{[n]}$ 
```

The activation functions $g(\cdot)$ are generally different at the final layer. Thus $g^{[2]}$ might be softmax for multinomial classification or sigmoid for binary classification, while ReLU or tanh might be the activation function $g(\cdot)$ at the internal layers.

The need for non-linear activation functions One of the reasons we use non-linear activation functions for each layer in a neural network is that if we did not, the resulting network is exactly equivalent to a single-layer network. Let's see why this is true. Imagine the first two layers of such a network of purely linear layers:

$$\begin{aligned}\mathbf{z}^{[1]} &= \mathbf{W}^{[1]}\mathbf{x} + \mathbf{b}^{[1]} \\ \mathbf{z}^{[2]} &= \mathbf{W}^{[2]}\mathbf{z}^{[1]} + \mathbf{b}^{[2]}\end{aligned}$$

We can rewrite the function that the network is computing as:

$$\begin{aligned}\mathbf{z}^{[2]} &= \mathbf{W}^{[2]}\mathbf{z}^{[1]} + \mathbf{b}^{[2]} \\ &= \mathbf{W}^{[2]}(\mathbf{W}^{[1]}\mathbf{x} + \mathbf{b}^{[1]}) + \mathbf{b}^{[2]} \\ &= \mathbf{W}^{[2]}\mathbf{W}^{[1]}\mathbf{x} + \mathbf{W}^{[2]}\mathbf{b}^{[1]} + \mathbf{b}^{[2]} \\ &= \mathbf{W}'\mathbf{x} + \mathbf{b}'\end{aligned}\tag{7.14}$$

This generalizes to any number of layers. So without non-linear activation functions, a multilayer network is just a notational variant of a single layer network with a different set of weights, and we lose all the representational power of multilayer networks.

Replacing the bias unit In describing networks, we will often use a slightly simplified notation that represents exactly the same function without referring to an explicit bias node b . Instead, we add a dummy node $\mathbf{a}_0$ to each layer whose value will always be 1. Thus layer 0, the input layer, will have a dummy node $\mathbf{a}_0^{[0]} = 1$, layer 1 will have $\mathbf{a}_0^{[1]} = 1$, and so on. This dummy node still has an associated weight, and that weight represents the bias value b . For example instead of an equation like

$$\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b}) \quad (7.15)$$

we'll use:

$$\mathbf{h} = \sigma(\mathbf{W}\mathbf{x}) \quad (7.16)$$

But now instead of our vector $\mathbf{x}$ having n_0 values: $\mathbf{x} = \mathbf{x}_1, \dots, \mathbf{x}_{n_0}$, it will have $n_0 + 1$ values, with a new 0th dummy value $\mathbf{x}_0 = 1$: $\mathbf{x} = \mathbf{x}_0, \dots, \mathbf{x}_{n_0}$. And instead of computing each $\mathbf{h}_j$ as follows:

$$\mathbf{h}_j = \sigma \left(\sum_{i=1}^{n_0} \mathbf{W}_{ji} \mathbf{x}_i + \mathbf{b}_j \right), \quad (7.17)$$

we'll instead use:

$$\mathbf{h}_j = \sigma \left(\sum_{i=0}^{n_0} \mathbf{W}_{ji} \mathbf{x}_i \right), \quad (7.18)$$

where the value $\mathbf{W}_{j0}$ replaces what had been $\mathbf{b}_j$. Fig. 7.9 shows a visualization.

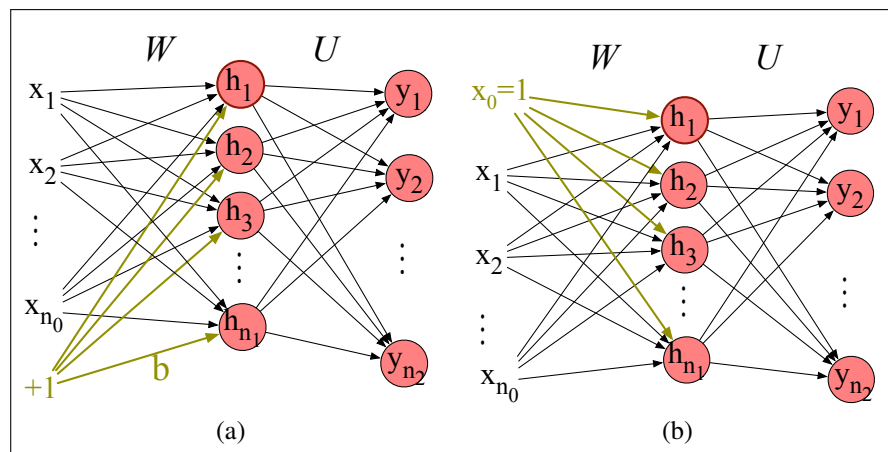


Figure 7.9 Replacing the bias node (shown in a) with x_0 (b).

We'll continue showing the bias as b when we go over the learning algorithm in Section 7.6, but then we'll switch to this simplified notation without explicit bias terms for the rest of the book.

7.4 Feedforward networks for NLP: Classification

Let's see how to apply feedforward networks to NLP tasks! In this section we'll look at classification tasks like sentiment analysis; in the next section we'll introduce neural language modeling.

Let's begin with a simple 2-layer sentiment classifier. You might imagine taking our logistic regression classifier from Chapter 5, which corresponds to a 1-layer network, and just adding a hidden layer. The input element $\mathbf{x}_i$ could be scalar features like those in Fig. 5.2, e.g., $\mathbf{x}_1 = \text{count}(\text{words} \in \text{doc})$, $\mathbf{x}_2 = \text{count}(\text{positive lexicon words} \in \text{doc})$, $\mathbf{x}_3 = 1$ if “no” $\in \text{doc}$, and so on. And the output layer $\hat{\mathbf{y}}$ could have two nodes (one each for positive and negative), or 3 nodes (positive, negative, neutral), in which case $\hat{y}_1$ would be the estimated probability of positive sentiment, $\hat{y}_2$ the probability of negative and $\hat{y}_3$ the probability of neutral. The resulting equations would be just what we saw above for a 2-layer network (as always, we'll continue to use the σ to stand for any non-linearity, whether sigmoid, ReLU or other).

$$\begin{aligned}\mathbf{x} &= [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N] \quad (\text{each } \mathbf{x}_i \text{ is a hand-designed feature}) \\ \mathbf{h} &= \sigma(\mathbf{W}\mathbf{x} + \mathbf{b}) \\ \mathbf{z} &= \mathbf{U}\mathbf{h} \\ \hat{\mathbf{y}} &= \text{softmax}(\mathbf{z})\end{aligned}\tag{7.19}$$

Fig. 7.10 shows a sketch of this architecture. As we mentioned earlier, adding this hidden layer to our logistic regression classifier allows the network to represent the non-linear interactions between features. This alone might give us a better sentiment classifier.

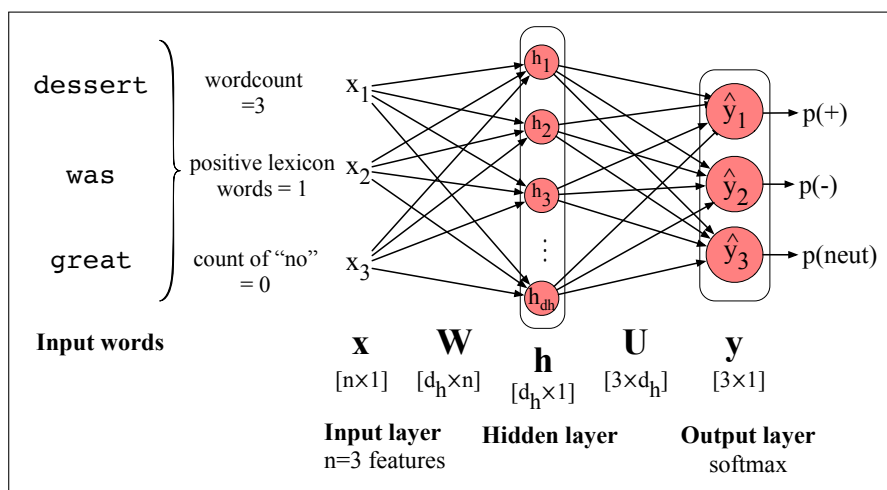


Figure 7.10 Feedforward network sentiment analysis using traditional hand-built features of the input text.

Most applications of neural networks for NLP do something different, however. Instead of using hand-built human-engineered features as the input to our classifier, we draw on deep learning's ability to learn features from the data by representing words as embeddings, like the word2vec or GloVe embeddings we saw in Chapter 6. There are various ways to represent an input for classification. One simple baseline is to apply some sort of **pooling** function to the embeddings of all the words in the input. For example, for a text with n input words/tokens $w_1, \dots, w_n$, we can turn the n embeddings $\mathbf{e}(w_1), \dots, \mathbf{e}(w_n)$ (each of dimensionality d) into a single embedding also of dimensionality d by just summing the embeddings, or by taking their mean (summing and then dividing by n):

$$\mathbf{x}_{\text{mean}} = \frac{1}{n} \sum_{i=1}^n \mathbf{e}(w_i)\tag{7.20}$$

There are many other options, like taking the element-wise max. The element-wise max of a set of n vectors is a new vector whose k th element is the max of the k th elements of all the n vectors. Here are the equations for this classifier assuming mean pooling; the architecture is sketched in Fig. 7.11:

$$\begin{aligned}
 \mathbf{x} &= \text{mean}(\mathbf{e}(w_1), \mathbf{e}(w_2), \dots, \mathbf{e}(w_n)) \\
 \mathbf{h} &= \sigma(\mathbf{W}\mathbf{x} + \mathbf{b}) \\
 \mathbf{z} &= \mathbf{U}\mathbf{h} \\
 \hat{\mathbf{y}} &= \text{softmax}(\mathbf{z})
 \end{aligned} \tag{7.21}$$

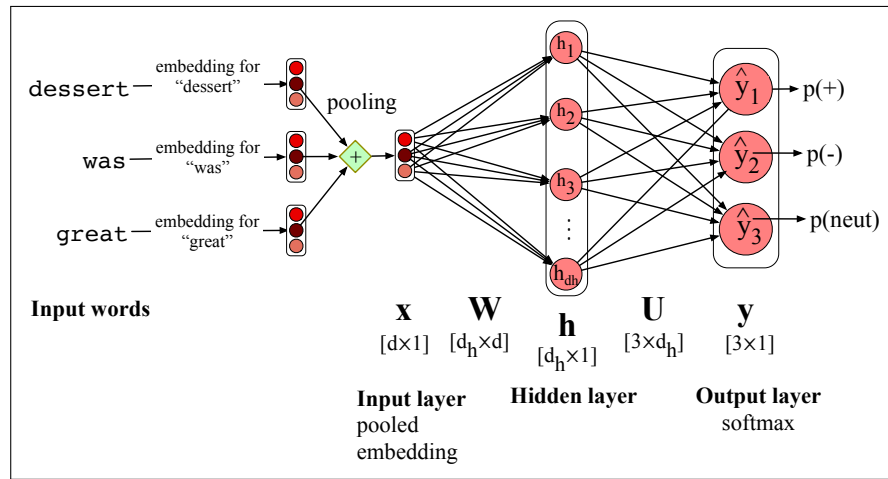


Figure 7.11 Feedforward network sentiment analysis using a pooled embedding of the input words.

While Eq. 7.21 shows how to classify a single example x , in practice we want to efficiently classify an entire test set of m examples. We do this by vectoring the process, just as we saw with logistic regression; instead of using for-loops to go through each example, we'll use matrix multiplication to do the entire computation of an entire test set at once. First, we pack all the input feature vectors for each input x into a single input matrix $\mathbf{X}$, with each row i a row vector consisting of the pooled embedding for input example $x^{(i)}$ (i.e., the vector $\mathbf{x}^{(i)}$). If the dimensionality of our pooled input embedding is d , $\mathbf{X}$ will be a matrix of shape $[m \times d]$.

We will then need to slightly modify Eq. 7.21. $\mathbf{X}$ is of shape $[m \times d]$ and $\mathbf{W}$ is of shape $[d_h \times d]$, so we'll have to reorder how we multiply $\mathbf{X}$ and $\mathbf{W}$ and transpose $\mathbf{W}$ so they correctly multiply to yield a matrix $\mathbf{H}$ of shape $[m \times d_h]$. The bias vector $\mathbf{b}$ from Eq. 7.21 of shape $[1 \times d_h]$ will now have to be replicated into a matrix of shape $[m \times d_h]$. We'll need to similarly reorder the next step and transpose $\mathbf{U}$. Finally, our output matrix $\hat{\mathbf{Y}}$ will be of shape $[m \times 3]$ (or more generally $[m \times d_o]$, where d_o is the number of output classes), with each row i of our output matrix $\hat{\mathbf{Y}}$ consisting of the output vector $\hat{\mathbf{y}}^{(i)}$. Here are the final equations for computing the output class distribution for an entire test set:

$$\begin{aligned}
 \mathbf{H} &= \sigma(\mathbf{X}\mathbf{W}^T + \mathbf{b}) \\
 \mathbf{Z} &= \mathbf{H}\mathbf{U}^T \\
 \hat{\mathbf{Y}} &= \text{softmax}(\mathbf{Z})
 \end{aligned} \tag{7.22}$$

pretraining The idea of using word2vec or GloVe embeddings as our input representation—and more generally the idea of relying on another algorithm to have already learned an embedding representation for our input words—is called **pretraining**. Using pretrained embedding representations, whether simple static word embeddings like word2vec or the much more powerful contextual embeddings we’ll introduce in Chapter 11, is one of the central ideas of deep learning. (It’s also possible, however, to train the word embeddings as part of an NLP task; we’ll talk about how to do this in Section 7.7 in the context of the neural language modeling task.)

7.5 Feedforward Neural Language Modeling

As our second application of feedforward networks, let’s consider **language modeling**: predicting upcoming words from prior word context. Neural language modeling is an important NLP task in itself, and it plays a role in many important algorithms for tasks like machine translation, summarization, speech recognition, grammar correction, and dialogue. We’ll describe simple feedforward neural language models, first introduced by [Bengio et al. \(2003\)](#). While modern neural language models use more powerful architectures like the recurrent nets or transformer networks to be introduced in Chapter 9, the feedforward language model introduces many of the important concepts of neural language modeling.

Neural language models have many advantages over the n-gram language models of Chapter 3. Compared to n-gram models, neural language models can handle much longer histories, can generalize better over contexts of similar words, and are more accurate at word-prediction. On the other hand, neural net language models are much more complex, are slower and need more energy to train, and are less interpretable than n-gram models, so for many (especially smaller) tasks an n-gram language model is still the right tool.

A feedforward neural language model (LM) is a feedforward network that takes as input at time t a representation of some number of previous words (w_{t-1}, w_{t-2} , etc.) and outputs a probability distribution over possible next words. Thus—like the n-gram LM—the feedforward neural LM approximates the probability of a word given the entire prior context $P(w_t | w_{1:t-1})$ by approximating based on the $N - 1$ previous words:

$$P(w_t | w_1, \dots, w_{t-1}) \approx P(w_t | w_{t-N+1}, \dots, w_{t-1}) \quad (7.23)$$

In the following examples we’ll use a 4-gram example, so we’ll show a neural net to estimate the probability $P(w_t = i | w_{t-3}, w_{t-2}, w_{t-1})$.

Neural language models represent words in this prior context by their **embeddings**, rather than just by their word identity as used in n-gram language models. Using embeddings allows neural language models to generalize better to unseen data. For example, suppose we’ve seen this sentence in training:

I have to make sure that the cat gets fed.

but have never seen the words “gets fed” after the word “dog”. Our test set has the prefix “I forgot to make sure that the dog gets”. What’s the next word? An n-gram language model will predict “fed” after “that the cat gets”, but not after “that the dog gets”. But a neural LM, knowing that “cat” and “dog” have similar embeddings, will be able to generalize from the “cat” context to assign a high enough probability to “fed” even after seeing “dog”.

7.5.1 Forward inference in the neural language model

forward
inference

Let's walk through **forward inference** or **decoding** for neural language models. Forward inference is the task, given an input, of running a forward pass on the network to produce a probability distribution over possible outputs, in this case next words.

one-hot vector

We first represent each of the N previous words as a one-hot vector of length $|V|$, i.e., with one dimension for each word in the vocabulary. A **one-hot vector** is a vector that has one element equal to 1—in the dimension corresponding to that word's index in the vocabulary—while all the other elements are set to zero. Thus in a one-hot representation for the word “toothpaste”, supposing it is V_5 , i.e., index 5 in the vocabulary, $x_5 = 1$, and $x_i = 0 \ \forall i \neq 5$, as shown here:

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 & \dots & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$\begin{matrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & \dots & \dots & |V| \end{matrix}$$

The feedforward neural language model (sketched in Fig. 7.13) has a moving window that can see N words into the past. We'll let N equal 3, so the 3 words w_{t-1} , w_{t-2} , and w_{t-3} are each represented as a one-hot vector. We then multiply these one-hot vectors by the embedding matrix $\mathbf{E}$. The embedding weight matrix $\mathbf{E}$ has a column for each word, each a column vector of d dimensions, and hence has dimensionality $d \times |V|$. Multiplying by a one-hot vector that has only one non-zero element $x_i = 1$ simply selects out the relevant column vector for word i , resulting in the embedding for word i , as shown in Fig. 7.12.

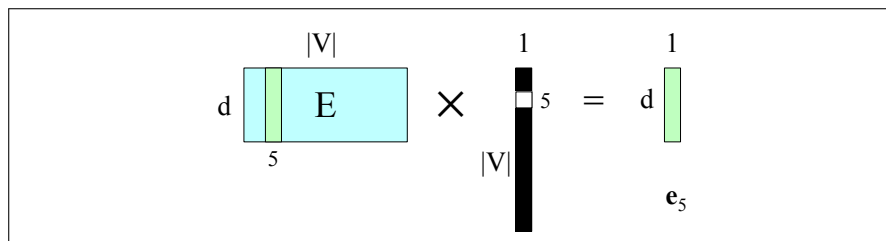


Figure 7.12 Selecting the embedding vector for word V_5 by multiplying the embedding matrix $\mathbf{E}$ with a one-hot vector with a 1 in index 5.

The 3 resulting embedding vectors are concatenated to produce $\mathbf{e}$, the embedding layer. This is followed by a hidden layer and an output layer whose softmax produces a probability distribution over words. For example y_{42} , the value of output node 42, is the probability of the next word w_t being V_{42} , the vocabulary word with index 42 (which is the word ‘fish’ in our example).

Here's the algorithm in detail for our mini example:

1. **Select three embeddings from $\mathbf{E}$:** Given the three previous words, we look up their indices, create 3 one-hot vectors, and then multiply each by the embedding matrix $\mathbf{E}$. Consider w_{t-3} . The one-hot vector for ‘for’ (index 35) is multiplied by the embedding matrix $\mathbf{E}$, to give the first part of the first hidden layer, the **embedding layer**. Since each column of the input matrix $\mathbf{E}$ is an embedding for a word, and the input is a one-hot column vector $\mathbf{x}_i$ for word V_i , the embedding layer for input w will be $\mathbf{E}\mathbf{x}_i = \mathbf{e}_i$, the embedding for word i . We now concatenate the three embeddings for the three context words to produce the embedding layer $\mathbf{e}$.
2. **Multiply by $\mathbf{W}$:** We multiply by $\mathbf{W}$ (and add b) and pass through the ReLU (or other) activation function to get the hidden layer h .

embedding
layer

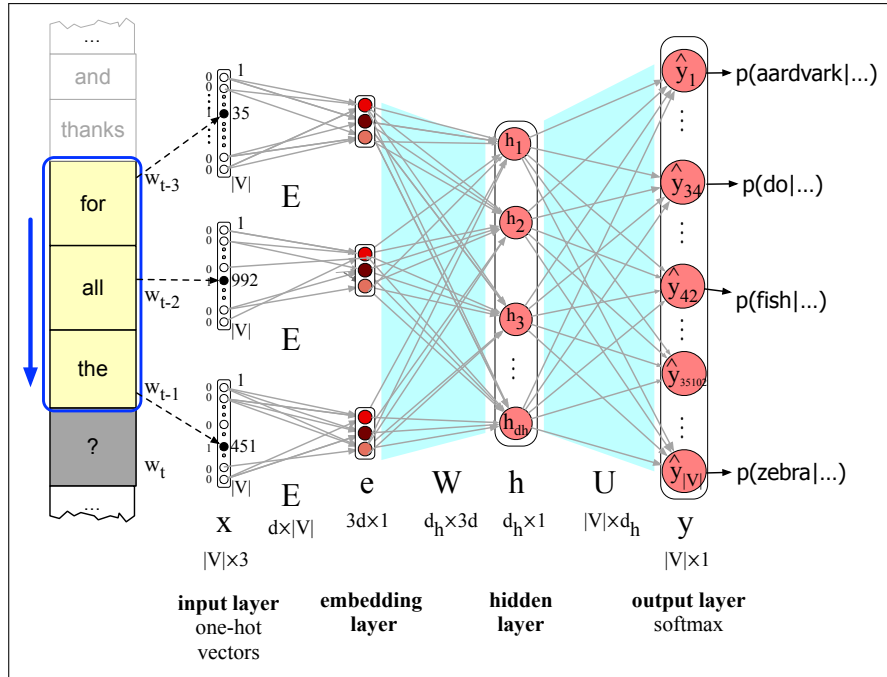


Figure 7.13 Forward inference in a feedforward neural language model. At each timestep t the network computes a d -dimensional embedding for each context word (by multiplying a one-hot vector by the embedding matrix E), and concatenates the 3 resulting embeddings to get the embedding layer e . The embedding vector e is multiplied by a weight matrix W and then an activation function is applied element-wise to produce the hidden layer h , which is then multiplied by another weight matrix U . Finally, a softmax output layer predicts at each node i the probability that the next word w_t will be vocabulary word V_i .

3. **Multiply by U:** h is now multiplied by U
4. **Apply softmax:** After the softmax, each node i in the output layer estimates the probability $P(w_t = i | w_{t-1}, w_{t-2}, w_{t-3})$

In summary, the equations for a neural language model with a window size of 3, given one-hot input vectors for each input context word, are:

$$\begin{aligned}
 e &= [Ex_{t-3}; Ex_{t-2}; Ex_{t-1}] \\
 h &= \sigma(We + b) \\
 z &= Uh \\
 \hat{y} &= \text{softmax}(z)
 \end{aligned} \tag{7.24}$$

Note that we formed the embedding layer e by concatenating the 3 embeddings for the three context vectors; we'll often use semicolons to mean concatenation of vectors.

In the next section we'll introduce a general algorithm for training neural networks, and then return to how to specifically train the neural language model in Section 7.7.

7.6 Training Neural Nets

A feedforward neural net is an instance of supervised machine learning in which we know the correct output y for each observation x . What the system produces, via Eq. 7.13, is $\hat{y}$, the system's estimate of the true y . The goal of the training procedure is to learn parameters $\mathbf{W}^{[i]}$ and $\mathbf{b}^{[i]}$ for each layer i that make $\hat{y}$ for each training observation as close as possible to the true y .

In general, we do all this by drawing on the methods we introduced in Chapter 5 for logistic regression, so the reader should be comfortable with that chapter before proceeding.

First, we'll need a **loss function** that models the distance between the system output and the gold output, and it's common to use the loss function used for logistic regression, the **cross-entropy loss**.

Second, to find the parameters that minimize this loss function, we'll use the **gradient descent** optimization algorithm introduced in Chapter 5.

Third, gradient descent requires knowing the **gradient** of the loss function, the vector that contains the partial derivative of the loss function with respect to each of the parameters. In logistic regression, for each observation we could directly compute the derivative of the loss function with respect to an individual w or b . But for neural networks, with millions of parameters in many layers, it's much harder to see how to compute the partial derivative of some weight in layer 1 when the loss is attached to some much later layer. How do we partial out the loss over all those intermediate layers? The answer is the algorithm called **error backpropagation** or **backward differentiation**.

7.6.1 Loss function

cross-entropy
loss

The **cross-entropy loss** that is used in neural networks is the same one we saw for logistic regression. If the neural network is being used as a binary classifier, with the sigmoid at the final layer, the loss function is the same logistic regression loss we saw in Eq. 5.22:

$$L_{CE}(\hat{y}, y) = -\log p(y|x) = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})] \quad (7.25)$$

If we are using the network to classify into 3 or more classes, the loss function is exactly the same as the loss for multinomial regression that we saw in Chapter 5 on page 99. Let's briefly summarize the explanation here for convenience. First, when we have more than 2 classes we'll need to represent both $\mathbf{y}$ and $\hat{\mathbf{y}}$ as vectors. Let's assume we're doing **hard classification**, where only one class is the correct one. The true label $\mathbf{y}$ is then a vector with K elements, each corresponding to a class, with $\mathbf{y}_c = 1$ if the correct class is c , with all other elements of $\mathbf{y}$ being 0. Recall that a vector like this, with one value equal to 1 and the rest 0, is called a **one-hot vector**. And our classifier will produce an estimate vector with K elements $\hat{\mathbf{y}}$, each element $\hat{y}_k$ of which represents the estimated probability $p(\mathbf{y}_k = 1|x)$.

The loss function for a single example $\mathbf{x}$ is the negative sum of the logs of the K output classes, each weighted by their probability y_k :

$$L_{CE}(\hat{\mathbf{y}}, \mathbf{y}) = -\sum_{k=1}^K \mathbf{y}_k \log \hat{y}_k \quad (7.26)$$

We can simplify this equation further; let's first rewrite the equation using the function $\mathbb{1}\{\}$ which evaluates to 1 if the condition in the brackets is true and to 0 oth-

erwise. This makes it more obvious that the terms in the sum in Eq. 7.26 will be 0 except for the term corresponding to the true class for which $\mathbf{y}_k = 1$:

$$L_{CE}(\hat{\mathbf{y}}, \mathbf{y}) = - \sum_{k=1}^K \mathbb{1}\{\mathbf{y}_k = 1\} \log \hat{\mathbf{y}}_k$$

negative log
likelihood loss

In other words, the cross-entropy loss is simply the negative log of the output probability corresponding to the correct class, and we therefore also call this the **negative log likelihood loss**:

$$L_{CE}(\hat{\mathbf{y}}, \mathbf{y}) = -\log \hat{\mathbf{y}}_c \quad (\text{where } c \text{ is the correct class}) \quad (7.27)$$

Plugging in the softmax formula from Eq. 7.9, and with K the number of classes:

$$L_{CE}(\hat{\mathbf{y}}, \mathbf{y}) = -\log \frac{\exp(\mathbf{z}_c)}{\sum_{j=1}^K \exp(\mathbf{z}_j)} \quad (\text{where } c \text{ is the correct class}) \quad (7.28)$$

7.6.2 Computing the Gradient

How do we compute the gradient of this loss function? Computing the gradient requires the partial derivative of the loss function with respect to each parameter. For a network with one weight layer and sigmoid output (which is what logistic regression is), we could simply use the derivative of the loss that we used for logistic regression in Eq. 7.29 (and derived in Section 5.10):

$$\begin{aligned} \frac{\partial L_{CE}(\hat{\mathbf{y}}, \mathbf{y})}{\partial w_j} &= (\hat{y} - y) \mathbf{x}_j \\ &= (\sigma(\mathbf{w} \cdot \mathbf{x} + b) - y) \mathbf{x}_j \end{aligned} \quad (7.29)$$

Or for a network with one weight layer and softmax output (=multinomial logistic regression), we could use the derivative of the softmax loss from Eq. 5.47, shown for a particular weight $\mathbf{w}_k$ and input $\mathbf{x}_i$

$$\begin{aligned} \frac{\partial L_{CE}(\hat{\mathbf{y}}, \mathbf{y})}{\partial \mathbf{w}_{k,i}} &= -(\mathbf{y}_k - \hat{\mathbf{y}}_k) \mathbf{x}_i \\ &= -(\mathbf{y}_k - p(\mathbf{y}_k = 1 | \mathbf{x})) \mathbf{x}_i \\ &= -\left(\mathbf{y}_k - \frac{\exp(\mathbf{w}_k \cdot \mathbf{x} + b_k)}{\sum_{j=1}^K \exp(\mathbf{w}_j \cdot \mathbf{x} + b_j)} \right) \mathbf{x}_i \end{aligned} \quad (7.30)$$

But these derivatives only give correct updates for one weight layer: the last one! For deep networks, computing the gradients for each weight is much more complex, since we are computing the derivative with respect to weight parameters that appear all the way back in the very early layers of the network, even though the loss is computed only at the very end of the network.

error back-
propagation

The solution to computing this gradient is an algorithm called **error backpropagation** or **backprop** (Rumelhart et al., 1986). While backprop was invented specially for neural networks, it turns out to be the same as a more general procedure called **backward differentiation**, which depends on the notion of **computation graphs**. Let's see how that works in the next subsection.

7.6.3 Computation Graphs

A computation graph is a representation of the process of computing a mathematical expression, in which the computation is broken down into separate operations, each of which is modeled as a node in a graph.

Consider computing the function $L(a, b, c) = c(a + 2b)$. If we make each of the component addition and multiplication operations explicit, and add names (d and e) for the intermediate outputs, the resulting series of computations is:

$$\begin{aligned} d &= 2 * b \\ e &= a + d \\ L &= c * e \end{aligned}$$

We can now represent this as a graph, with nodes for each operation, and directed edges showing the outputs from each operation as the inputs to the next, as in Fig. 7.14. The simplest use of computation graphs is to compute the value of the function with some given inputs. In the figure, we've assumed the inputs $a = 3$, $b = 1$, $c = -2$, and we've shown the result of the **forward pass** to compute the result $L(3, 1, -2) = -10$. In the forward pass of a computation graph, we apply each operation left to right, passing the outputs of each computation as the input to the next node.

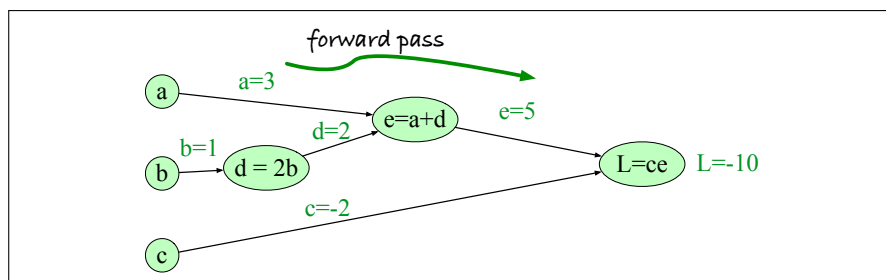


Figure 7.14 Computation graph for the function $L(a, b, c) = c(a + 2b)$, with values for input nodes $a = 3$, $b = 1$, $c = -2$, showing the forward pass computation of L .

7.6.4 Backward differentiation on computation graphs

The importance of the computation graph comes from the **backward pass**, which is used to compute the derivatives that we'll need for the weight update. In this example our goal is to compute the derivative of the output function L with respect to each of the input variables, i.e., $\frac{\partial L}{\partial a}$, $\frac{\partial L}{\partial b}$, and $\frac{\partial L}{\partial c}$. The derivative $\frac{\partial L}{\partial a}$ tells us how much a small change in a affects L .

chain rule

Backwards differentiation makes use of the **chain rule** in calculus, so let's remind ourselves of that. Suppose we are computing the derivative of a composite function $f(x) = u(v(x))$. The derivative of $f(x)$ is the derivative of $u(x)$ with respect to $v(x)$ times the derivative of $v(x)$ with respect to x :

$$\frac{df}{dx} = \frac{du}{dv} \cdot \frac{dv}{dx} \quad (7.31)$$

The chain rule extends to more than two functions. If computing the derivative of a composite function $f(x) = u(v(w(x)))$, the derivative of $f(x)$ is:

$$\frac{df}{dx} = \frac{du}{dv} \cdot \frac{dv}{dw} \cdot \frac{dw}{dx} \quad (7.32)$$

The intuition of backward differentiation is to pass gradients back from the final node to all the nodes in the graph. Fig. 7.15 shows part of the backward computation at one node e . Each node takes an upstream gradient that is passed in from its parent node to the right, and for each of its inputs computes a local gradient (the gradient of its output with respect to its input), and uses the chain rule to multiply these two to compute a downstream gradient to be passed on to the next earlier node.

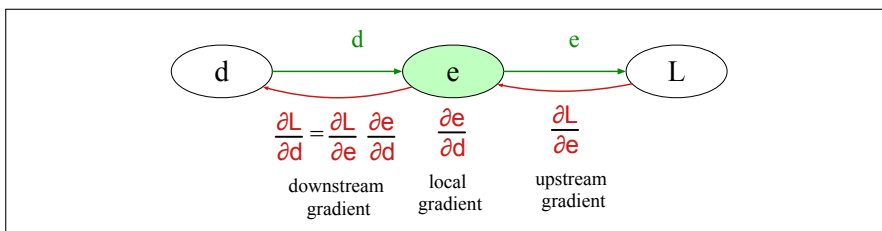


Figure 7.15 Each node (like e here) takes an upstream gradient, multiplies it by the local gradient (the gradient of its output with respect to its input), and uses the chain rule to compute a downstream gradient to be passed on to a prior node. A node may have multiple local gradients if it has multiple inputs.

Let's now compute the 3 derivatives we need. Since in the computation graph $L = ce$, we can directly compute the derivative $\frac{\partial L}{\partial c}$:

$$\frac{\partial L}{\partial c} = e \quad (7.33)$$

For the other two, we'll need to use the chain rule:

$$\begin{aligned} \frac{\partial L}{\partial a} &= \frac{\partial L}{\partial e} \frac{\partial e}{\partial a} \\ \frac{\partial L}{\partial b} &= \frac{\partial L}{\partial e} \frac{\partial e}{\partial d} \frac{\partial d}{\partial b} \end{aligned} \quad (7.34)$$

Eq. 7.34 and Eq. 7.33 thus require five intermediate derivatives: $\frac{\partial L}{\partial e}$, $\frac{\partial L}{\partial c}$, $\frac{\partial e}{\partial a}$, $\frac{\partial e}{\partial d}$, and $\frac{\partial d}{\partial b}$, which are as follows (making use of the fact that the derivative of a sum is the sum of the derivatives):

$$\begin{aligned} L = ce & : \quad \frac{\partial L}{\partial e} = c, \frac{\partial L}{\partial c} = e \\ e = a + d & : \quad \frac{\partial e}{\partial a} = 1, \frac{\partial e}{\partial d} = 1 \\ d = 2b & : \quad \frac{\partial d}{\partial b} = 2 \end{aligned}$$

In the backward pass, we compute each of these partials along each edge of the graph from right to left, using the chain rule just as we did above. Thus we begin by computing the downstream gradients from node L , which are $\frac{\partial L}{\partial e}$ and $\frac{\partial L}{\partial c}$. For node e , we then multiply this upstream gradient $\frac{\partial L}{\partial e}$ by the local gradient (the gradient of the output with respect to the input), $\frac{\partial e}{\partial d}$ to get the output we send back to node d : $\frac{\partial L}{\partial d}$. And so on, until we have annotated the graph all the way to all the input variables. The forward pass conveniently already will have computed the values of the forward intermediate variables we need (like d and e) to compute these derivatives. Fig. 7.16 shows the backward pass.

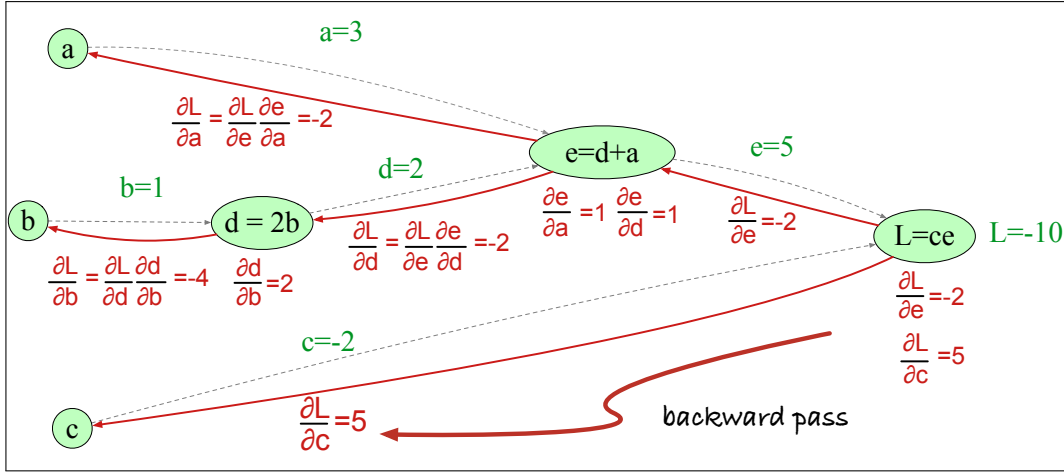


Figure 7.16 Computation graph for the function $L(a, b, c) = c(a + 2b)$, showing the backward pass computation of $\frac{\partial L}{\partial a}$, $\frac{\partial L}{\partial b}$, and $\frac{\partial L}{\partial c}$.

Backward differentiation for a neural network

Of course computation graphs for real neural networks are much more complex. Fig. 7.17 shows a sample computation graph for a 2-layer neural network with $n_0 = 2$, $n_1 = 2$, and $n_2 = 1$, assuming binary classification and hence using a sigmoid output unit for simplicity. The function that the computation graph is computing is:

$$\begin{aligned}
 \mathbf{z}^{[1]} &= \mathbf{W}^{[1]}\mathbf{x} + \mathbf{b}^{[1]} \\
 \mathbf{a}^{[1]} &= \text{ReLU}(\mathbf{z}^{[1]}) \\
 \mathbf{z}^{[2]} &= \mathbf{W}^{[2]}\mathbf{a}^{[1]} + \mathbf{b}^{[2]} \\
 a^{[2]} &= \sigma(\mathbf{z}^{[2]}) \\
 \hat{y} &= a^{[2]}
 \end{aligned} \tag{7.35}$$

For the backward pass we'll also need to compute the loss L . The loss function for binary sigmoid output from Eq. 7.25 is

$$L_{CE}(\hat{y}, y) = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})] \tag{7.36}$$

Our output $\hat{y} = a^{[2]}$, so we can rephrase this as

$$L_{CE}(a^{[2]}, y) = -[y \log a^{[2]} + (1 - y) \log(1 - a^{[2]})] \tag{7.37}$$

The weights that need updating (those for which we need to know the partial derivative of the loss function) are shown in teal. In order to do the backward pass, we'll need to know the derivatives of all the functions in the graph. We already saw in Section 5.10 the derivative of the sigmoid σ :

$$\frac{d\sigma(z)}{dz} = \sigma(z)(1 - \sigma(z)) \tag{7.38}$$

We'll also need the derivatives of each of the other activation functions. The derivative of tanh is:

$$\frac{d \tanh(z)}{dz} = 1 - \tanh^2(z) \tag{7.39}$$

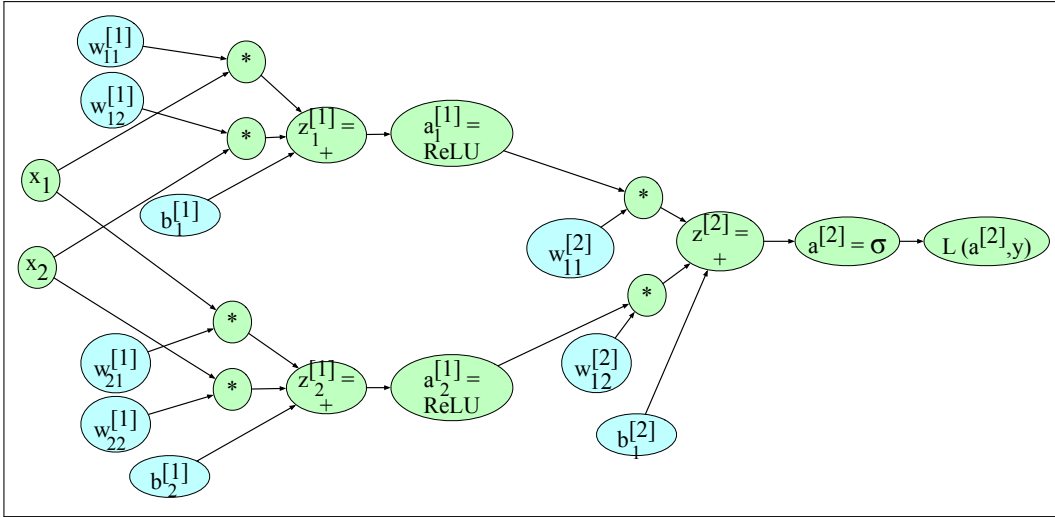


Figure 7.17 Sample computation graph for a simple 2-layer neural net (= 1 hidden layer) with two input units and 2 hidden units. We’ve adjusted the notation a bit to avoid long equations in the nodes by just mentioning the function that is being computed, and the resulting variable name. Thus the * to the right of node $w_{11}^{[1]}$ means that $w_{11}^{[1]}$ is to be multiplied by x_1 , and the node $z_1^{[1]} = +$ means that the value of $z_1^{[1]}$ is computed by summing the three nodes that feed into it (the two products, and the bias term $b_1^{[1]}$).

The derivative of the ReLU is

$$\frac{d\text{ReLU}(z)}{dz} = \begin{cases} 0 & \text{for } z < 0 \\ 1 & \text{for } z \geq 0 \end{cases} \quad (7.40)$$

We’ll give the start of the computation, computing the derivative of the loss function L with respect to z , or $\frac{\partial L}{\partial z}$ (and leaving the rest of the computation as an exercise for the reader). By the chain rule:

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial a^{[2]}} \frac{\partial a^{[2]}}{\partial z} \quad (7.41)$$

So let’s first compute $\frac{\partial L}{\partial a^{[2]}}$, taking the derivative of Eq. 7.37, repeated here:

$$\begin{aligned} L_{CE}(a^{[2]}, y) &= -[y \log a^{[2]} + (1 - y) \log(1 - a^{[2]})] \\ \frac{\partial L}{\partial a^{[2]}} &= -\left(\left(y \frac{\partial \log(a^{[2]})}{\partial a^{[2]}} \right) + (1 - y) \frac{\partial \log(1 - a^{[2]})}{\partial a^{[2]}} \right) \\ &= -\left(\left(y \frac{1}{a^{[2]}} \right) + (1 - y) \frac{1}{1 - a^{[2]}} (-1) \right) \\ &= -\left(\frac{y}{a^{[2]}} + \frac{y - 1}{1 - a^{[2]}} \right) \end{aligned} \quad (7.42)$$

Next, by the derivative of the sigmoid:

$$\frac{\partial a^{[2]}}{\partial z} = a^{[2]}(1 - a^{[2]})$$

Finally, we can use the chain rule:

$$\begin{aligned}
 \frac{\partial L}{\partial z} &= \frac{\partial L}{\partial a^{[2]}} \frac{\partial a^{[2]}}{\partial z} \\
 &= -\left(\frac{y}{a^{[2]}} + \frac{y-1}{1-a^{[2]}}\right) a^{[2]}(1-a^{[2]}) \\
 &= a^{[2]} - y
 \end{aligned} \tag{7.43}$$

Continuing the backward computation of the gradients (next by passing the gradients over $b_1^{[2]}$ and the two product nodes, and so on, back to all the teal nodes), is left as an exercise for the reader.

7.6.5 More details on learning

Optimization in neural networks is a non-convex optimization problem, more complex than for logistic regression, and for that and other reasons there are many best practices for successful learning.

For logistic regression we can initialize gradient descent with all the weights and biases having the value 0. In neural networks, by contrast, we need to initialize the weights with small random numbers. It's also helpful to normalize the input values to have 0 mean and unit variance.

Various forms of regularization are used to prevent overfitting. One of the most important is **dropout**: randomly dropping some units and their connections from the network during training (Hinton et al. 2012, Srivastava et al. 2014). Tuning of **hyperparameters** is also important. The parameters of a neural network are the weights W and biases b ; those are learned by gradient descent. The hyperparameters are things that are chosen by the algorithm designer; optimal values are tuned on a devset rather than by gradient descent learning on the training set. Hyperparameters include the learning rate η , the mini-batch size, the model architecture (the number of layers, the number of hidden nodes per layer, the choice of activation functions), how to regularize, and so on. Gradient descent itself also has many architectural variants such as Adam (Kingma and Ba, 2015).

Finally, most modern neural networks are built using computation graph formalisms that make it easy and natural to do gradient computation and parallelization on vector-based GPUs (Graphic Processing Units). PyTorch (Paszke et al., 2017) and TensorFlow (Abadi et al., 2015) are two of the most popular. The interested reader should consult a neural network textbook for further details; some suggestions are at the end of the chapter.

7.7 Training the neural language model

Now that we've seen how to train a generic neural net, let's talk about the architecture for training a neural language model, setting the parameters $\theta = \mathbf{E}, \mathbf{W}, \mathbf{U}, \mathbf{b}$.

For some tasks, it's ok to **freeze** the embedding layer $\mathbf{E}$ with initial word2vec values. Freezing means we use word2vec or some other pretraining algorithm to compute the initial embedding matrix $\mathbf{E}$, and then hold it constant while we only modify $\mathbf{W}$, $\mathbf{U}$, and $\mathbf{b}$, i.e., we don't update $\mathbf{E}$ during language model training. However, often we'd like to learn the embeddings simultaneously with training the network. This is useful when the task the network is designed for (like sentiment clas-

sification, translation, or parsing) places strong constraints on what makes a good representation for words.

Let's see how to train the entire model including $\mathbf{E}$, i.e. to set all the parameters $\theta = \mathbf{E}, \mathbf{W}, \mathbf{U}, \mathbf{b}$. We'll do this via gradient descent (Fig. 5.6), using error backpropagation on the computation graph to compute the gradient. Training thus not only sets the weights $\mathbf{W}$ and $\mathbf{U}$ of the network, but also as we're predicting upcoming words, we're learning the embeddings $\mathbf{E}$ for each word that best predict upcoming words.

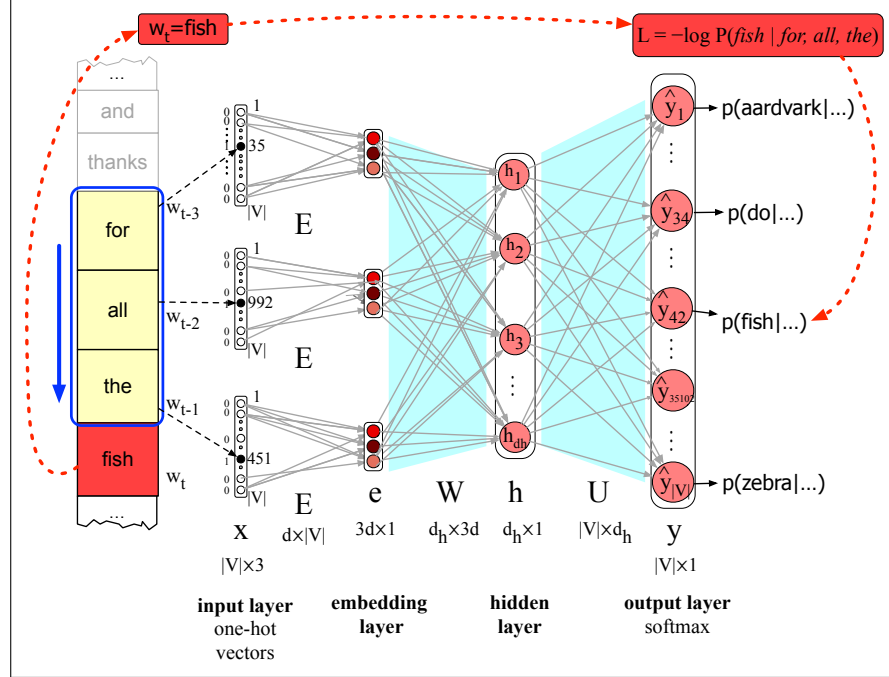


Figure 7.18 Learning all the way back to embeddings. Again, the embedding matrix $\mathbf{E}$ is shared among the 3 context words.

Fig. 7.18 shows the set up for a window size of $N=3$ context words. The input $\mathbf{x}$ consists of 3 one-hot vectors, fully connected to the embedding layer via 3 instantiations of the embedding matrix $\mathbf{E}$. We don't want to learn separate weight matrices for mapping each of the 3 previous words to the projection layer. We want one single embedding dictionary $\mathbf{E}$ that's shared among these three. That's because over time, many different words will appear as w_{t-2} or w_{t-1} , and we'd like to just represent each word with one vector, whichever context position it appears in. Recall that the embedding weight matrix $\mathbf{E}$ has a column for each word, each a column vector of d dimensions, and hence has dimensionality $d \times |V|$.

Generally training proceeds by taking as input a very long text, concatenating all the sentences, starting with random weights, and then iteratively moving through the text predicting each word w_t . At each word w_t , we use the cross-entropy (negative log likelihood) loss. Recall that the general form for this (repeated from Eq. 7.27 is:

$$L_{CE}(\hat{y}, y) = -\log \hat{y}_i, \quad (\text{where } i \text{ is the correct class}) \quad (7.44)$$

For language modeling, the classes are the words in the vocabulary, so $\hat{y}_i$ here means the probability that the model assigns to the correct next word w_t :

$$L_{CE} = -\log p(w_t | w_{t-1}, \dots, w_{t-n+1}) \quad (7.45)$$

The parameter update for stochastic gradient descent for this loss from step s to $s + 1$ is then:

$$\theta^{s+1} = \theta^s - \eta \frac{\partial [-\log p(w_t | w_{t-1}, \dots, w_{t-n+1})]}{\partial \theta} \quad (7.46)$$

This gradient can be computed in any standard neural network framework which will then backpropagate through $\theta = \mathbf{E}, \mathbf{W}, \mathbf{U}, \mathbf{b}$.

Training the parameters to minimize loss will result both in an algorithm for language modeling (a word predictor) but also a new set of embeddings $\mathbf{E}$ that can be used as word representations for other tasks.

7.8 Summary

- Neural networks are built out of **neural units**, originally inspired by human neurons but now simply an abstract computational device.
- Each neural unit multiplies input values by a weight vector, adds a bias, and then applies a non-linear activation function like sigmoid, tanh, or rectified linear unit.
- In a **fully-connected, feedforward** network, each unit in layer i is connected to each unit in layer $i + 1$, and there are no cycles.
- The power of neural networks comes from the ability of early layers to learn representations that can be utilized by later layers in the network.
- Neural networks are trained by optimization algorithms like **gradient descent**.
- **Error backpropagation**, backward differentiation on a **computation graph**, is used to compute the gradients of the loss function for a network.
- **Neural language models** use a neural network as a probabilistic classifier, to compute the probability of the next word given the previous n words.
- Neural language models can use pretrained **embeddings**, or can learn embeddings from scratch in the process of language modeling.

Bibliographical and Historical Notes

The origins of neural networks lie in the 1940s **McCulloch-Pitts neuron** (McCulloch and Pitts, 1943), a simplified model of the human neuron as a kind of computing element that could be described in terms of propositional logic. By the late 1950s and early 1960s, a number of labs (including Frank Rosenblatt at Cornell and Bernard Widrow at Stanford) developed research into neural networks; this phase saw the development of the perceptron (Rosenblatt, 1958), and the transformation of the threshold into a bias, a notation we still use (Widrow and Hoff, 1960).

The field of neural networks declined after it was shown that a single perceptron unit was unable to model functions as simple as XOR (Minsky and Papert, 1969). While some small amount of work continued during the next two decades, a major revival for the field didn't come until the 1980s, when practical tools for building deeper networks like error backpropagation became widespread (Rumelhart et al.,

connectionist

1986). During the 1980s a wide variety of neural network and related architectures were developed, particularly for applications in psychology and cognitive science (Rumelhart and McClelland 1986b, McClelland and Elman 1986, Rumelhart and McClelland 1986a, Elman 1990), for which the term **connectionist** or **parallel distributed processing** was often used (Feldman and Ballard 1982, Smolensky 1988). Many of the principles and techniques developed in this period are foundational to modern work, including the ideas of distributed representations (Hinton, 1986), recurrent networks (Elman, 1990), and the use of tensors for compositionality (Smolensky, 1990).

By the 1990s larger neural networks began to be applied to many practical language processing tasks as well, like handwriting recognition (LeCun et al. 1989) and speech recognition (Morgan and Bourlard 1990). By the early 2000s, improvements in computer hardware and advances in optimization and training techniques made it possible to train even larger and deeper networks, leading to the modern term **deep learning** (Hinton et al. 2006, Bengio et al. 2007). We cover more related history in Chapter 9 and Chapter 16.

There are a number of excellent books on the subject. Goldberg (2017) has superb coverage of neural networks for natural language processing. For neural networks in general see Goodfellow et al. (2016) and Nielsen (2015).